

Analysis of Algorithms

Concise Study Notes: Chapters 1–7

ENCT 388 Year III, Part II

Scope. These notes follow the full syllabus of Analysis of Algorithms (ENCT 388, 45 hours, 60 theory marks): Chapter 1 Introduction to Algorithm Analysis, Chapter 2 Iterative and Numeric Algorithms, Chapter 3 Divide and Conquer, Chapter 4 Greedy Algorithms, Chapter 5 Dynamic Programming, Chapter 6 Backtracking, Chapter 7 NP-Hard and NP-Complete Problems. Every numerical is worked step by step and every derivation is shown. Chapters 2, 4 and 6 follow the lecture notes of Er. Sujan Karki; all lecture assignments and practice questions are solved here. A complexity summary and 30 MCQs with an answer key are at the end.

Contents

	Chapter	Topics	Page
1	Introduction to Algorithm Analysis	RAM model; time/space complexity; aggregate analysis; O , Ω , Θ ; best/average/worst case; recurrences; recursion tree, substitution, master theorem	2
2	Iterative and Numeric Algorithms	GCD, Fibonacci, sequential search, elementary sorts; Euclid and extended Euclid; CRT; modular equations; Fermat; Miller–Rabin	8
3	Divide and Conquer	Binary search; min–max; merge, heap, quick, randomized quick sort; order statistics in expected and worst-case linear time	15
4	Greedy Algorithms	Greedy strategy; fractional knapsack; job sequencing; Kruskal, Prim; Dijkstra	21
5	Dynamic Programming	Floyd–Warshall; TSP; edit distance; 0/1 knapsack; matrix-chain; flow-shop scheduling	27
6	Backtracking	N -Queens; sum of subsets; graph colouring; Hamiltonian cycle; 0/1 knapsack	33
7	NP-Hard and NP-Complete Problems	P, NP, reductions, Cook’s theorem; NP-hard graph, scheduling and code-generation problems; approximation algorithms	38
	Summary and knowledge check	Complexity table; 30 MCQs with answer key	43

Chapter 1 Introduction to Algorithm Analysis

1.1 Algorithm, RAM Model and Complexity

Algorithm

An **algorithm** is a finite sequence of well-defined, unambiguous instructions that transforms a given *input* into the desired *output* in a finite amount of time. *Example*: sorting a list, finding the GCD of two integers, finding a shortest path.

Properties of an algorithm. (1) **Input**: zero or more externally supplied quantities. (2) **Output**: at least one result. (3) **Definiteness**: every step is precisely and unambiguously stated. (4) **Finiteness**: it terminates after a finite number of steps for every valid input. (5) **Effectiveness**: every step is basic enough to be carried out exactly in finite time. (6) **Correctness**: it produces the right output for every valid input.

Why analyse? Analysis predicts the resources (time, memory) an algorithm needs, *independent of machine, language and compiler*, so that competing algorithms can be compared and the best chosen before any code is written.

RAM (Random Access Machine) model

The analysis machine is a hypothetical **RAM**: instructions are executed one after another with *no concurrency*; every simple operation (arithmetic $+$, $-$, $\times$, $\div$, comparison, assignment, array indexing, branch, return) takes exactly *one unit* of time; every memory access (read/write of one word) takes one unit; there is unlimited memory and no memory hierarchy (cache is ignored). Loops and procedure calls are *not* single operations: their cost is the sum of the operations they execute.

Time and space complexity

Time complexity $T(n)$ is the number of primitive operations executed as a function of the **input size** n . **Space complexity** $S(n)$ is the memory used as a function of n : input space + *auxiliary* (extra) space. An algorithm using $O(1)$ extra space is called **in-place**.

Input size n is: the number of elements (sorting, searching); the number of bits (numeric algorithms such as GCD or primality); $|V| + |E|$ (graph algorithms).

Detailed analysis of algorithms

Example: **Sum of an array**.

Statement	Cost	Times
sum = 0	c_1	1
for i = 1 to n	c_2	$n + 1$
sum = sum + A[i]	c_3	n
return sum	c_4	1

$T(n) = c_1 + c_2(n + 1) + c_3n + c_4 = (c_2 + c_3)n + (c_1 + c_2 + c_4) = an + b$: a linear function, so $T(n) = \Theta(n)$.

Example: **Insertion sort, full line-by-line cost**. Let t_j be the number of times the `while` test is executed for index j .

Statement	Cost	Times
for j = 2 to n	c_1	n
key = A[j]	c_2	$n - 1$
i = j - 1	c_3	$n - 1$
while i > 0 and A[i] > key	c_4	$\sum_{j=2}^n t_j$
A[i+1] = A[i]	c_5	$\sum_{j=2}^n (t_j - 1)$
i = i - 1	c_6	$\sum_{j=2}^n (t_j - 1)$
A[i+1] = key	c_7	$n - 1$

$$T(n) = c_1 n + (c_2 + c_3 + c_7)(n - 1) + c_4 \sum_{j=2}^n t_j + (c_5 + c_6) \sum_{j=2}^n (t_j - 1).$$

Best case (already sorted): $t_j = 1$, so the two sums are $n - 1$ and 0, giving $T(n) = an + b$: **linear**.

Worst case (reverse sorted): $t_j = j$, so $\sum_{j=2}^n j = \frac{n(n+1)}{2} - 1$ and $\sum_{j=2}^n (j - 1) = \frac{n(n-1)}{2}$; hence

$$T(n) = \frac{c_4 + c_5 + c_6}{2} n^2 + \left(c_1 + c_2 + c_3 + c_7 + \frac{c_4 - c_5 - c_6}{2} \right) n - (c_2 + c_3 + c_4 + c_7) = an^2 + bn + c : \text{quadratic.}$$

Counting loops: standard patterns

Loop pattern	Count	Complexity
for i=1..n	n	$O(n)$
for i=1..n: for j=1..n	$n \cdot n$	$O(n^2)$
for i=1..n: for j=1..i	$1 + 2 + \dots + n = \frac{n(n+1)}{2}$	$O(n^2)$
for (i=1; i<n; i=i*2)	$\lceil \log_2 n \rceil$	$O(\log n)$
for (i=1; i<n; i=i*2): for j=1..i	$1 + 2 + 4 + \dots + 2^k, 2^k < n, < 2n$	$O(n)$
for i=1..n: for (j=1; j<n; j=j*2)	$n \log_2 n$	$O(n \log n)$

Aggregate analysis

Aggregate analysis

For a *sequence* of n operations, bound the *total* worst-case cost $T(n)$ of the whole sequence, then say each operation has **amortized cost** $T(n)/n$. A single operation may be expensive, but the total is small because expensive operations are rare. It is a worst-case guarantee over the sequence (no probability involved).

Example: Stack with MULTIPOP. PUSH, POP cost 1; MULTIPOP(k) pops $\min(k, s)$ items. A single MULTIPOP can cost $O(n)$, so a naive bound for n operations is $O(n^2)$. But each element can be popped at most once for each time it was pushed, so the total number of pops (including those inside MULTIPOP) $\leq$ number of pushes $\leq n$. Total cost $\leq 2n$, amortized cost per operation = $2n/n = O(1)$.

Example: k-bit binary counter, INCREMENT. Bit $A[0]$ flips every call, $A[1]$ every 2nd call, $A[i]$ every 2^i -th call. For n increments the total flips are

$$\sum_{i=0}^{k-1} \left\lfloor \frac{n}{2^i} \right\rfloor < n \sum_{i=0}^{\infty} \frac{1}{2^i} = 2n,$$

so the amortized cost per increment is $O(1)$, although one increment (e.g. 0111 $\rightarrow$ 1000) may flip k bits.

1.2 Asymptotic Notations

We compare growth rates for *large* n , ignoring constant factors and lower-order terms.

Big-O, Big-Ω, Big-Θ (formal definitions)

$f(n) = O(g(n))$ iff there exist constants $c > 0$, $n_0 \geq 1$ with $0 \leq f(n) \leq c g(n)$ for all $n \geq n_0$. (upper bound)

$f(n) = \Omega(g(n))$ iff there exist $c > 0$, n_0 with $0 \leq c g(n) \leq f(n)$ for all $n \geq n_0$. (lower bound)

$f(n) = \Theta(g(n))$ iff there exist $c_1, c_2 > 0$, n_0 with $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$. (tight bound)

Equivalently $f = \Theta(g) \iff f = O(g)$ and $f = \Omega(g)$. Little-o: $f = o(g)$ if $\lim f/g = 0$ (strictly smaller); little-ω: $f = \omega(g)$ if $\lim f/g = \infty$.

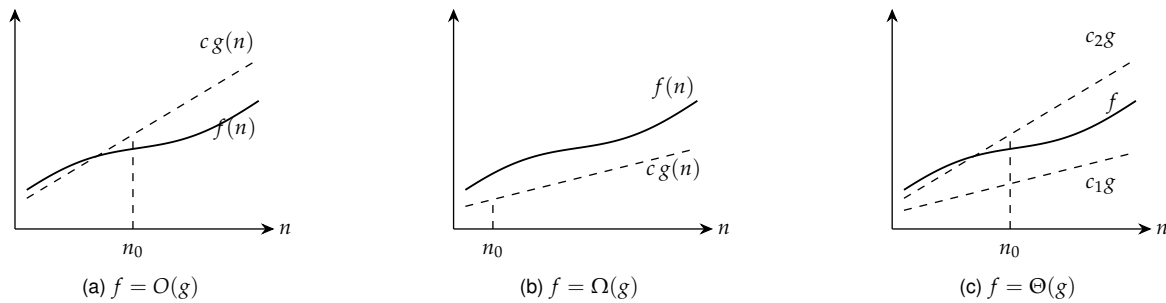


Figure 1: Geometric interpretation: beyond n_0 , f stays below $c g$ (a), above $c g$ (b), or sandwiched between $c_1 g$ and $c_2 g$ (c).

Worked proofs (find c and n_0).

- $3n + 2 = O(n)$: $3n + 2 \leq 3n + n = 4n$ for $n \geq 2$. So $c = 4$, $n_0 = 2$.
- $3n + 2 = \Omega(n)$: $3n + 2 \geq 3n$ for all $n \geq 1$. So $c = 3$, $n_0 = 1$. Hence $3n + 2 = \Theta(n)$.
- $10n^2 + 4n + 2 = O(n^2)$: for $n \geq 1$, $4n \leq 4n^2$ and $2 \leq 2n^2$, so $f \leq 16n^2$. $c = 16$, $n_0 = 1$; also $f \geq 10n^2$, so $\Theta(n^2)$.
- $n^2 \neq O(n)$ (proof by contradiction): suppose $n^2 \leq cn$ for all $n \geq n_0$; then $n \leq c$ for all large n , which is false.
- $2^{n+1} = O(2^n)$: $2^{n+1} = 2 \cdot 2^n$, so $c = 2$, $n_0 = 1$. But $2^{2n} \neq O(2^n)$ since $2^{2n}/2^n = 2^n \rightarrow \infty$.
- $\frac{1}{2}n^2 - 3n = \Theta(n^2)$: need $c_1 n^2 \leq \frac{1}{2}n^2 - 3n \leq c_2 n^2$. Dividing by n^2 : $c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2$. Take $c_2 = \frac{1}{2}$; for $n \geq 7$, $\frac{1}{2} - \frac{3}{n} \geq \frac{1}{14}$, so $c_1 = \frac{1}{14}$, $n_0 = 7$.
- $\log n! = \Theta(n \log n)$: $\log n! = \sum \log i \leq n \log n$; and $\log n! \geq \sum_{i=n/2}^n \log i \geq \frac{n}{2} \log \frac{n}{2} = \Omega(n \log n)$.

Limit test. Let $L = \lim_{n \rightarrow \infty} f(n)/g(n)$. If $L = 0$: $f = o(g)$ (so O , not Ω). If $0 < L < \infty$: $f = \Theta(g)$. If $L = \infty$: $f = \omega(g)$ (so Ω , not O). Example: $\lim n \log n / n^2 = \lim \log n / n = 0$, so $n \log n = o(n^2)$.

Properties.

Reflexive	$f = O(f)$, $f = \Omega(f)$, $f = \Theta(f)$.
Transitive	$f = O(g)$ and $g = O(h) \Rightarrow f = O(h)$ (same for Ω, Θ).
Symmetric	$f = \Theta(g) \iff g = \Theta(f)$.
Transpose symmetry	$f = O(g) \iff g = \Omega(f)$.
Sum rule	$O(f) + O(g) = O(\max(f, g))$; so in a sequence of statements the costliest dominates.
Product rule	$O(f) \cdot O(g) = O(f \cdot g)$; used for nested loops.
Polynomials	$a_d n^d + \dots + a_0 = \Theta(n^d)$ for $a_d > 0$.
Logs	$\log_a n = \Theta(\log_b n)$ for constants $a, b > 1$ (change of base); so the base is ignored in $O(\cdot)$.

Order of growth (slowest to fastest):

$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n! < n^n.$$

n	$\log_2 n$	$n \log_2 n$	n^2	n^3	2^n	$n!$
4	2	8	16	64	16	24
8	3	24	64	512	256	40 320
16	4	64	256	4 096	65 536	$\approx 2.1 \times 10^{13}$
32	5	160	1 024	32 768	$\approx 4.3 \times 10^9$	$\approx 2.6 \times 10^{35}$

1.3 Best, Average and Worst Case

Cases of performance

For inputs of size n : **best case** $B(n) = \min T$ (the input on which the algorithm is fastest); **worst case** $W(n) = \max T$ (an upper bound guaranteed for every input; most commonly used); **average case** $A(n) = \sum_I P(I) T(I)$, the expected time under a stated probability distribution of inputs.

Example: Linear search for a key in an array of n distinct elements. Best: key at position 1, 1 comparison, $\Theta(1)$. Worst: key last or absent, n comparisons, $\Theta(n)$. Average (key present, each position equally likely):

$$A(n) = \frac{1}{n} \sum_{i=1}^n i = \frac{n+1}{2} = \Theta(n).$$

If the key is present with probability p and absent otherwise, $A(n) = p \frac{n+1}{2} + (1-p)n$.

Note. Best/worst/average are *cases of the input*; O, Ω, Θ are *bounds on a function*. Either can be applied to any case: “insertion sort is $\Omega(n)$ in the best case and $O(n^2)$ in the worst case”.

1.4 Modeling Algorithms by Recurrence Relations

A **recurrence** expresses $T(n)$ in terms of T of smaller inputs plus the non-recursive work. For a divide-and-conquer algorithm:

$$T(n) = a T\left(\frac{n}{b}\right) + f(n),$$

where $a \geq 1$ is the number of subproblems, n/b the size of each ($b > 1$), and $f(n)$ the cost of dividing and combining.

Algorithm	Recurrence	Solution	Reason
Factorial (recursive)	$T(n) = T(n-1) + 1$	$\Theta(n)$	one call, $O(1)$ work
Binary search	$T(n) = T(n/2) + 1$	$\Theta(\log n)$	halve, one compare
Merge sort	$T(n) = 2T(n/2) + n$	$\Theta(n \log n)$	two halves, linear merge
Tower of Hanoi	$T(n) = 2T(n-1) + 1$	$\Theta(2^n)$	two sub-towers
Fibonacci (naive)	$T(n) = T(n-1) + T(n-2) + 1$	$\Theta(\phi^n)$	two overlapping calls
Quick sort worst	$T(n) = T(n-1) + n$	$\Theta(n^2)$	unbalanced split
Quick sort best	$T(n) = 2T(n/2) + n$	$\Theta(n \log n)$	balanced split
Max-min (D&C)	$T(n) = 2T(n/2) + 2$	$\Theta(n)$	two halves, $O(1)$ combine

Example: Tower of Hanoi, expansion. $T(n) = 2T(n-1) + 1 = 2^2T(n-2) + 2 + 1 = \dots = 2^{n-1}T(1) + (2^{n-1} - 1) = 2^n - 1$ with $T(1) = 1$.

1.5 Solving Recurrence Relations

Method 1: Recursion-tree method

Draw the tree of recursive calls, write the work done at each node, sum the work *level by level*, then sum over all levels.

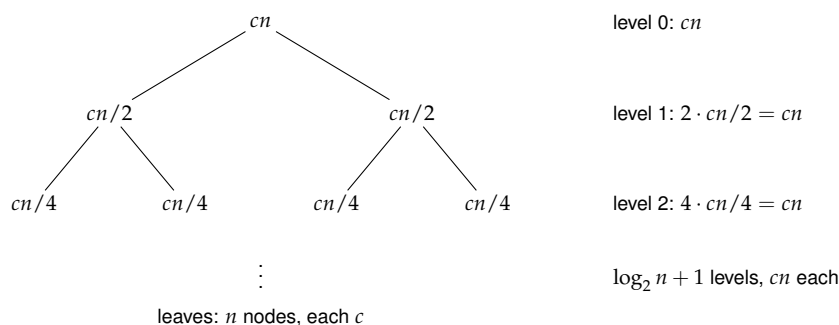


Figure 2: Recursion tree for $T(n) = 2T(n/2) + cn$: every level costs cn .

Example: $T(n) = 2T(n/2) + cn$. Each level costs cn and there are $\log_2 n + 1$ levels, so $T(n) = cn \log_2 n + cn = \Theta(n \log n)$.

Example: $T(n) = 3T(n/4) + cn^2$. Level i has 3^i nodes each of size $n/4^i$, costing $3^i c (n/4^i)^2 = (3/16)^i cn^2$. The tree depth is $\log_4 n$ and the leaves number $3^{\log_4 n} = n^{\log_4 3}$. Sum of internal levels (geometric, ratio $3/16 < 1$):

$$T(n) \leq cn^2 \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i + \Theta(n^{\log_4 3}) = \frac{16}{13} cn^2 + \Theta(n^{0.79}) = \Theta(n^2).$$

Example: Unbalanced tree: $T(n) = T(n/3) + T(2n/3) + cn$. Every full level costs $\leq cn$; the longest path follows the $2n/3$ branch and has length $\log_{3/2} n$. So $T(n) = O(cn \log_{3/2} n) = O(n \log n)$ (and $\Omega(n \log n)$ since the shortest path, of length $\log_3 n$, is full to that depth).

Method 2: Substitution method

Steps: (1) *guess* the form of the solution (from the tree or from experience); (2) assume it holds for smaller $m < n$ (induction hypothesis) and substitute into the recurrence; (3) show it holds for n and choose constants; (4) check the base case.

Example: Show $T(n) = 2T(\lfloor n/2 \rfloor) + n = O(n \log n)$. Guess $T(n) \leq cn \log_2 n$. Assume it for $\lfloor n/2 \rfloor$:

$$T(n) \leq 2c \lfloor n/2 \rfloor \log_2 \lfloor n/2 \rfloor + n \leq cn \log_2 (n/2) + n = cn \log_2 n - cn + n \leq cn \log_2 n \quad (c \geq 1).$$

Base case: $T(1) = 1$ but $c \cdot 1 \cdot \log_2 1 = 0$, so $n = 1$ fails; use $n_0 = 2$: with $T(2) = 4$ and $T(3) = 5$ as bases, $c \geq 2$ works ($2 \cdot 2 \cdot 1 = 4 \geq 4$; $2 \cdot 3 \cdot \log_2 3 \approx 9.5 \geq 5$). Hence $T(n) = O(n \log n)$.

Example: Subtracting a lower-order term. $T(n) = 2T(n/2) + 1$. Guess $T(n) \leq cn$: $T(n) \leq 2c(n/2) + 1 = cn + 1$, which does *not* prove $T(n) \leq cn$. Strengthen the hypothesis: $T(n) \leq cn - d$:

$$T(n) \leq 2(c \frac{n}{2} - d) + 1 = cn - 2d + 1 \leq cn - d \quad \text{if } d \geq 1.$$

So $T(n) = O(n)$.

Example: Change of variable. $T(n) = 2T(\sqrt{n}) + \log_2 n$. Put $m = \log_2 n$, $n = 2^m$: $T(2^m) = 2T(2^{m/2}) + m$. Let $S(m) = T(2^m)$: $S(m) = 2S(m/2) + m \Rightarrow S(m) = \Theta(m \log m)$. Hence $T(n) = \Theta(\log n \log \log n)$.

Method 3: Master theorem**Master theorem**

Let $T(n) = aT(n/b) + f(n)$ with $a \geq 1$, $b > 1$, f asymptotically positive. Compare $f(n)$ with $n^{\log_b a}$ (the cost of the leaves):

1. **Case 1.** If $f(n) = O(n^{\log_b a - \varepsilon})$ for some $\varepsilon > 0$ (leaves dominate): $T(n) = \Theta(n^{\log_b a})$.
2. **Case 2.** If $f(n) = \Theta(n^{\log_b a} \log^k n)$ with $k \geq 0$ (every level equal): $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$. For $k = 0$: $T(n) = \Theta(n^{\log_b a} \log n)$.
3. **Case 3.** If $f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some $\varepsilon > 0$ and the regularity condition $af(n/b) \leq cf(n)$ holds for some $c < 1$ (root dominates): $T(n) = \Theta(f(n))$.

Procedure: identify $a, b, f(n)$; compute $n^{\log_b a}$; compare polynomially with $f(n)$; apply the matching case.

Recurrence	a	b	$n^{\log_b a}$	Case	Result
$T(n) = 9T(n/3) + n$	9	3	n^2	1	$\Theta(n^2)$
$T(n) = 8T(n/2) + n^2$	8	2	n^3	1	$\Theta(n^3)$
$T(n) = 2T(n/2) + n$	2	2	n	2	$\Theta(n \log n)$
$T(n) = 4T(n/2) + n^2$	4	2	n^2	2	$\Theta(n^2 \log n)$
$T(n) = T(2n/3) + 1$	1	3/2	$n^0 = 1$	2	$\Theta(\log n)$
$T(n) = T(n/2) + 1$	1	2	1	2	$\Theta(\log n)$
$T(n) = 2T(n/2) + n^2$	2	2	n	3	$\Theta(n^2)$
$T(n) = 3T(n/4) + n \log n$	3	4	$n^{0.793}$	3	$\Theta(n \log n)$
$T(n) = 2T(n/2) + n \log n$	2	2	n	2 ($k=1$)	$\Theta(n \log^2 n)$

Example: Case 3 regularity check for $T(n) = 3T(n/4) + n \log n$: $af(n/b) = 3 \frac{n}{4} \log \frac{n}{4} \leq \frac{3}{4} n \log n = cf(n)$ with $c = \frac{3}{4} < 1$. ✓

Example: **Master theorem does not apply:** $T(n) = 2T(n/2) + n/\log n$ (here f is smaller than n but not *polynomially* smaller, and $k = -1 < 0$); $T(n) = 2^n T(n/2) + n^n$ (a not constant); $T(n) = 0.5 T(n/2) + n$ ($a < 1$); $T(n) = T(n-1) + n$ (not of the form n/b) – use the tree or substitution: $T(n) = n + (n-1) + \dots + 1 = \Theta(n^2)$.

Chapter 1 recap. RAM model, count primitive steps; O is an upper bound, Ω a lower bound, Θ a tight bound; recurrences from divide-and-conquer are $T(n) = aT(n/b) + f(n)$ and are solved by (i) recursion tree, (ii) substitution (guess and verify), (iii) the master theorem comparing $f(n)$ with $n^{\log_b a}$. Aggregate analysis bounds the total cost of n operations and divides by n .

Chapter 2 Iterative and Numeric Algorithms

An **iterative algorithm** repeats a set of instructions with a loop (for, while, do-while) until a condition is met, moving step by step towards the result. Unlike recursion it does not call itself, so it needs no call-stack.

2.1 GCD and Fibonacci Number

Algorithm for GCD(a, b) (iterative Euclid)

Algorithm: GCD

1. If $a = 0$ and $b = 0$: GCD is undefined. If $b = 0$ output $|a|$; if $a = 0$ output $|b|$.
2. While $b \neq 0$: $r \leftarrow a \bmod b$; $a \leftarrow b$; $b \leftarrow r$.
3. Output $|a|$.

```
def gcd(a, b):
    if a == 0 and b == 0: return None # undefined
    while b != 0:
        a, b = b, a % b
    return abs(a)
```

Example: Trace for gcd(270, 192).

Step	a	b	$r = a \bmod b$
Initial	270	192	–
1	192	78	$270 \bmod 192 = 78$
2	78	36	$192 \bmod 78 = 36$
3	36	6	$78 \bmod 36 = 6$
4	6	0	$36 \bmod 6 = 0$

The loop stops when $b = 0$; gcd(270, 192) = 6.

Time complexity (derivation). Each iteration maps $(a, b) \rightarrow (b, r)$ with $r = a \bmod b < b$, so the second number strictly decreases and the algorithm terminates. The worst case is when the remainders shrink as slowly as possible: consecutive Fibonacci numbers. If $a = F_n, b = F_{n-1}$ then $F_n \bmod F_{n-1} = F_{n-2}$, so $(F_n, F_{n-1}) \rightarrow (F_{n-1}, F_{n-2}) \rightarrow \dots$, i.e. the index drops by one per step: about n steps. Using $F_k \approx \phi^k / \sqrt{5}$ with $\phi = \frac{1+\sqrt{5}}{2} \approx 1.618$ and $b = F_{n-1} \approx \phi^{n-1} / \sqrt{5}$:

$$\begin{aligned} \log_{\phi} b &\approx (n-1) - \log_{\phi} \sqrt{5} \Rightarrow n \approx \log_{\phi} b + 1 + \log_{\phi} \sqrt{5} \approx \log_{\phi} b \\ &= \frac{\log_2 b}{\log_2 \phi} = \frac{\log_2 b}{0.694} \approx 1.44 \log_2 b. \end{aligned}$$

Hence $T(n) = O(\log \min(a, b))$. **Space:** only the variable r is used, $S(n) = O(1)$. Example: gcd(F_{36}, F_{35}): $14\,930\,352 \bmod 9\,227\,465 = 5\,702\,887 = F_{34}$ – the remainder is the next Fibonacci number, confirming the worst case.

Fibonacci series

Fibonacci series

$F(0) = 0, F(1) = 1, F(n) = F(n-1) + F(n-2)$ for $n \geq 2$: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Iterative algorithm: keep the last two values. Initialise prev = 0, curr = 1, $i = 2$; while $i \leq n$: next = prev + curr, prev = curr, curr = next, $i = i + 1$; return curr.


```
def fibonacci(n):          #  $F(0)=0, F(1)=1$ 
    if n == 0: return 0
    prev, curr = 0, 1
    for i in range(2, n + 1):
        prev, curr = curr, prev + curr
    return curr
```

Example: Trace for $n = 5$ (so $F(5) = 5$).

Step	prev	curr	next
Initial	0	1	–
$i = 2$	1	1	$0 + 1 = 1$
$i = 3$	1	2	$1 + 1 = 2$
$i = 4$	2	3	$1 + 2 = 3$
$i = 5$	3	5	$2 + 3 = 5$

Complexity. The loop body does one addition and a few assignments, $O(1)$, and runs $n - 1$ times: $T(n) = O(n)$; only the variables `prev`, `curr`, `next`, `i` are used: $S(n) = O(1)$. (Naive recursion $F(n - 1) + F(n - 2)$ costs $\Theta(\phi^n)$ because subproblems are recomputed.)

2.2 Sequential (Linear) Search

Linear search

Compare the key with each element one by one, from the first, until it is found or the list ends. Works on sorted and unsorted data.

```
def linear_search(A, n, key):
    for i in range(n):
        if A[i] == key:
            return i          # found at index i
    return -1                # not found
```

Case	Condition	Comparisons	Complexity
Best	key at first position	1	$O(1)$
Worst	key last or absent	n	$O(n)$
Average	key equally likely at any position	$\frac{1}{n} \sum_{i=1}^n i = \frac{n+1}{2} \approx \frac{n}{2}$	$O(n)$

Space complexity $O(1)$ (variables i and a flag only).

Example: Search 20 in $[12, 5, 10, 15, 31, 20, 25, 2, 40]$: compare with 12, 5, 10, 15, 31 (no), then $20 = 20$ at index 5 after **6 comparisons**.

2.3 Review of Elementary Sorting Algorithms

Bubble sort

Bubble sort

Repeatedly compare adjacent elements and swap them if $A[j] > A[j + 1]$. After each pass the largest unsorted element moves (“bubbles”) to its final position at the end.

```
def bubble_sort(A):
    n = len(A)
```

```

for i in range(n):
    for j in range(0, n - i - 1):
        if A[j] > A[j + 1]:
            A[j], A[j + 1] = A[j + 1], A[j]
return A

```

Example: **Trace**, $A = [5, 1, 4, 2, 8]$.

Pass 1 $[5, 1, 4, 2, 8] \rightarrow [1, 5, 4, 2, 8] \rightarrow [1, 4, 5, 2, 8] \rightarrow [1, 4, 2, 5, 8] \rightarrow [1, 4, 2, 5, 8]$ (3 swaps; 8 in place)
 Pass 2 $[1, 4, 2, 5, 8] \rightarrow [1, 4, 2, 5, 8] \rightarrow [1, 2, 4, 5, 8] \rightarrow [1, 2, 4, 5, 8]$ (1 swap; 5 in place)
 Pass 3 $[1, 2, 4, 5, 8]$ no swap – sorted (an early-exit flag stops here)

Analysis. Comparisons are $(n - 1) + (n - 2) + \dots + 1 = \frac{n(n-1)}{2}$ regardless of input (without early exit).

- *Best case* (sorted, with early exit): one pass of $n - 1$ comparisons, no swaps: $O(n)$.
- *Worst case* (reverse sorted): every comparison swaps: comparisons + swaps = $\frac{n(n-1)}{2} + \frac{n(n-1)}{2} = n(n - 1) = O(n^2)$.
- *Average case*: about half the comparisons swap: $\frac{n(n-1)}{2} + \frac{n(n-1)}{4} = \frac{3}{4}(n^2 - n) = O(n^2)$.
- Space $O(1)$ (in-place); stable.

Selection sort

Selection sort

Find the minimum of the unsorted part and swap it into the first unsorted position; the sorted prefix grows by one each pass. It performs at most $n - 1$ swaps.

```

def selection_sort(A):
    n = len(A)
    for i in range(n - 1):
        m = i
        for j in range(i + 1, n):
            if A[j] < A[m]: m = j
        if m != i: A[i], A[m] = A[m], A[i]
    return A

```

Example: **Trace**, $A = [64, 25, 12, 22, 11]$. $i=0$: min = 11, swap $\rightarrow [11, 25, 12, 22, 64]$; $i=1$: min = 12, swap $\rightarrow [11, 12, 25, 22, 64]$; $i=2$: min = 22, swap $\rightarrow [11, 12, 22, 25, 64]$; $i=3$: min = 25 already in place. Sorted.

Analysis. Comparisons are always $\frac{n(n-1)}{2}$, so best = average = worst = $\Theta(n^2)$; swaps $\leq n - 1$; space $O(1)$; not stable.

Insertion sort

Insertion sort

Treat $A[0]$ as sorted. For $i = 1, \dots, n - 1$ take $temp = A[i]$ and shift larger elements of the sorted prefix one place right, then insert $temp$ in the gap (like sorting playing cards).

```

def insertion_sort(A):
    for i in range(1, len(A)):
        temp = A[i]
        j = i - 1
        while j >= 0 and A[j] > temp:
            A[j + 1] = A[j]
            j -= 1
        A[j + 1] = temp
    return A

```

Example: Trace, $A = [5, 2, 4, 6, 1, 3]$. $i=1$: insert 2 $\rightarrow [2, 5, 4, 6, 1, 3]$; $i=2$: insert 4 $\rightarrow [2, 4, 5, 6, 1, 3]$; $i=3$: 6 stays; $i=4$: insert 1 $\rightarrow [1, 2, 4, 5, 6, 3]$; $i=5$: insert 3 $\rightarrow [1, 2, 3, 4, 5, 6]$.

Analysis. Best case (sorted): inner loop never runs, $n - 1$ comparisons, 0 shifts: $O(n)$. Worst case (reverse sorted): comparisons $= \frac{n(n-1)}{2}$, shifts $= \frac{n(n-1)}{2}$, total $n(n-1) = O(n^2)$. Average: about half the shifts, $\frac{n(n-1)}{2} + \frac{n(n-1)}{4} = \frac{3n(n-1)}{4} = O(n^2)$. Space $O(1)$; stable; excellent for small or nearly sorted arrays.

Sort	Best	Average	Worst	Space	Stable
Bubble (early exit)	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	no
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	yes

2.4 Number-Theoretic Notations

Concept	Notation	Meaning / example
Sets	$\mathbb{N}, \mathbb{N}_0, \mathbb{Z}, \mathbb{Q}, \mathbb{R}, \mathbb{C}, \mathbb{P}$	naturals $\{1, 2, \dots\}$, whole numbers, integers, rationals, reals, complex, primes $\{2, 3, 5, 7, \dots\}$
Divisibility	$a \mid b$	a divides b with no remainder: $3 \mid 12$; $5 \nmid 12$
Congruence	$a \equiv b \pmod{m}$	a, b leave the same remainder on division by m (equivalently $m \mid (a - b)$): $17 \equiv 5 \pmod{12}$, $23 \equiv 3 \pmod{10}$
GCD	$\gcd(a, b)$	largest integer dividing both: $\gcd(48, 18) = 6$; $\gcd(a, 0) = a$
LCM	$\text{lcm}(a, b)$	smallest positive common multiple; $\text{lcm}(4, 6) = 12$; $\gcd(a, b) \cdot \text{lcm}(a, b) = ab$
Prime / composite		prime $p > 1$ has exactly two divisors 1, p ; composite $n > 1$ has more
Coprime	$\gcd(a, b) = 1$	no common divisor except 1: 8, 15 are coprime
Modular inverse	$a^{-1} \pmod{m}$	x with $ax \equiv 1 \pmod{m}$; exists iff $\gcd(a, m) = 1$

Properties of congruence: if $a \equiv b$ and $c \equiv d \pmod{m}$ then $a + c \equiv b + d$, $a - c \equiv b - d$, $ac \equiv bd$ and $a^k \equiv b^k \pmod{m}$.

2.5 Euclid's and Extended Euclid's Algorithms

Euclid's algorithm

Based on $\gcd(a, b) = \gcd(b, a \bmod b)$ and $\gcd(a, 0) = a$: repeatedly replace the larger number by the remainder of dividing it by the smaller, until the remainder is 0; the last non-zero remainder is the GCD. *Proof of the identity:* any common divisor of a, b divides $a - qb = a \bmod b$, and any common divisor of $b, a \bmod b$ divides $qb + (a \bmod b) = a$; the sets of common divisors coincide.

Recursive version: $\gcd(a, b) = a$ if $b == 0$ else $\gcd(b, a \bmod b)$. Time $O(\log \min(a, b))$; each call stores a frame, so space $O(\log \min(a, b))$ (the iterative form uses $O(1)$).

Example: $\gcd(30, 21) \rightarrow \gcd(21, 9) \rightarrow \gcd(9, 3) \rightarrow \gcd(3, 0) = 3$.

Extended Euclidean algorithm

Besides $d = \gcd(a, b)$ it finds integers x, y (possibly zero or negative) with $ax + by = d$ (Bézout's identity).

Recursion: if $b = 0$ return $(a, 1, 0)$. Otherwise compute $(d, x_1, y_1) = \text{ExtGCD}(b, a \bmod b)$ and return $(d, x = y_1, y = x_1 - \lfloor a/b \rfloor y_1)$.

Why: $bx_1 + (a \bmod b)y_1 = d$ and $a \bmod b = a - \lfloor a/b \rfloor b$, so $d = ay_1 + b(x_1 - \lfloor a/b \rfloor y_1)$.

```
def extended_gcd(a, b):
    if b == 0:
        return (a, 1, 0)
    d, x1, y1 = extended_gcd(b, a % b)
    return (d, y1, x1 - (a // b) * y1)
```

Complexity: the number of calls is that of Euclid's algorithm, so time and stack space are both $O(\log \min(a, b))$.

Example: Find x, y with $161x + 28y = \gcd(161, 28)$. $\text{ExtGCD}(161, 28) \rightarrow (28, 161 \bmod 28 = 21) \rightarrow (21, 28 \bmod 21 = 7) \rightarrow (7, 21 \bmod 7 = 0) \rightarrow (7, 1, 0)$. Back-substitute:

- $(21, 7)$: $x = y_1 = 0, y = x_1 - \lfloor 21/7 \rfloor y_1 = 1 - 3 \cdot 0 = 1 \rightarrow (7, 0, 1)$.
- $(28, 21)$: $x = 1, y = 0 - \lfloor 28/21 \rfloor \cdot 1 = -1 \rightarrow (7, 1, -1)$.
- $(161, 28)$: $x = -1, y = 1 - \lfloor 161/28 \rfloor (-1) = 1 + 5 = 6 \rightarrow (7, -1, 6)$.

Check: $161(-1) + 28(6) = -161 + 168 = 7 = \gcd$. ✓

Example: (**Question 2.2**) $\gcd(198, 128)$ and x, y with $198x + 128y = \gcd$. Division chain:

$$198 = 1 \cdot 128 + 70, \quad 128 = 1 \cdot 70 + 58, \quad 70 = 1 \cdot 58 + 12, \quad 58 = 4 \cdot 12 + 10, \quad 12 = 1 \cdot 10 + 2, \quad 10 = 5 \cdot 2 + 0.$$

So $\gcd(198, 128) = 2$. Back-substitution (express each remainder through 198 and 128):

$$\begin{aligned} 2 &= 12 - 10 = 12 - (58 - 4 \cdot 12) = 5 \cdot 12 - 58 \\ &= 5(70 - 58) - 58 = 5 \cdot 70 - 6 \cdot 58 = 5 \cdot 70 - 6(128 - 70) = 11 \cdot 70 - 6 \cdot 128 \\ &= 11(198 - 128) - 6 \cdot 128 = 11 \cdot 198 - 17 \cdot 128. \end{aligned}$$

Thus $x = 11, y = -17$. Check: $11 \cdot 198 = 2178, 17 \cdot 128 = 2176$, difference = 2. ✓

Why the extended algorithm is used (Question 2.2.2). It writes the GCD as a linear combination $ax + by$. Applications: (i) **modular inverse**: if $\gcd(a, m) = 1$ then $ax + my = 1 \Rightarrow ax \equiv 1 \pmod{m}$, so $a^{-1} = x \bmod m$; (ii) **RSA**: private key $d = e^{-1} \bmod \phi(n)$; (iii) solving linear Diophantine equations $ax + by = c$ (solvable iff $\gcd(a, b) \mid c$); (iv) the Chinese remainder theorem and modular linear equations (below); (v) ECDSA, Diffie–Hellman, key generation and signature verification.

2.6 Chinese Remainder Theorem and Modular Linear Equations

Chinese Remainder Theorem (CRT)

If $m_1, \dots, m_k$ are **pairwise coprime** ($\gcd(m_i, m_j) = 1, i \neq j$), then the system $x \equiv a_i \pmod{m_i}, i = 1..k$, has a *unique* solution modulo $M = m_1 m_2 \cdots m_k$ (Sun Tzu, *Sunzi Suanjing*, 3rd century).

Method. (1) $M = \prod m_i$. (2) $M_i = M/m_i$. (3) find y_i with $M_i y_i \equiv 1 \pmod{m_i}$ (modular inverse, via trial or extended Euclid). (4) $x = (\sum a_i M_i y_i) \bmod M$.

Why it works: $M_i \equiv 0 \pmod{m_j}$ for $j \neq i$ and $M_i y_i \equiv 1 \pmod{m_i}$, so the term $a_i M_i y_i$ is $\equiv a_i \pmod{m_i}$ and $\equiv 0$ modulo every other modulus.

Example: Solve $x \equiv 2 \pmod{3}, x \equiv 3 \pmod{5}, x \equiv 2 \pmod{7}$.

$$M = 105; M_1 = 35, M_2 = 21, M_3 = 15.$$

$$35y_1 \equiv 1 \pmod{3}: 35 \equiv 2, 2y_1 \equiv 1 \Rightarrow y_1 = 2. \quad 21y_2 \equiv 1 \pmod{5}: 21 \equiv 1 \Rightarrow y_2 = 1. \quad 15y_3 \equiv 1 \pmod{7}: 15 \equiv 1 \Rightarrow y_3 = 1.$$

$$x = (2 \cdot 35 \cdot 2 + 3 \cdot 21 \cdot 1 + 2 \cdot 15 \cdot 1) \bmod 105 = (140 + 63 + 30) \bmod 105 = 233 \bmod 105 = 23.$$

Verify: $23 \bmod 3 = 2, 23 \bmod 5 = 3, 23 \bmod 7 = 2$. ✓

Example: (**Assignment 2.3**) $x \equiv 1 \pmod{2}, x \equiv 2 \pmod{3}, x \equiv 3 \pmod{5}$. $M = 30; M_1 = 15, M_2 = 10, M_3 = 6$. $15y_1 \equiv 1 \pmod{2} \Rightarrow y_1 = 1; 10y_2 \equiv 1 \pmod{3} (10 \equiv 1) \Rightarrow y_2 = 1; 6y_3 \equiv 1 \pmod{5}$

$(6 \equiv 1) \Rightarrow y_3 = 1$. $x = (1 \cdot 15 + 2 \cdot 10 + 3 \cdot 6) \bmod 30 = 53 \bmod 30 = 23$. Check: $23 \bmod 2 = 1$, $23 \bmod 3 = 2$, $23 \bmod 5 = 3$. ✓

Example: (Assignment) $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$. $M = 15$; $M_1 = 5, M_2 = 3$; $5y_1 \equiv 1 \pmod{3} \Rightarrow 2y_1 \equiv 1 \Rightarrow y_1 = 2$; $3y_2 \equiv 1 \pmod{5} \Rightarrow y_2 = 2$. $x = (2 \cdot 5 \cdot 2 + 3 \cdot 3 \cdot 2) \bmod 15 = 38 \bmod 15 = 8$. Check: $8 \bmod 3 = 2$, $8 \bmod 5 = 3$. ✓

Solving modular linear equations $ax \equiv b \pmod{m}$

General solution method

1. $d = \gcd(a, m)$. 2. If $d \nmid b$: **no solution**. If $d \mid b$ there are **exactly d solutions** modulo m .
2. Divide by d : $\frac{a}{d}x \equiv \frac{b}{d} \pmod{m/d}$, now $\gcd(a/d, m/d) = 1$.
3. Find the inverse inv of a/d modulo m/d (extended Euclid or trial).
4. $x_0 = \frac{b}{d} \cdot inv \bmod \frac{m}{d}$.
5. All solutions: $x = x_0 + t \cdot \frac{m}{d}$, $t = 0, 1, \dots, d - 1$.

Example: $14x \equiv 30 \pmod{100}$. $d = \gcd(14, 100) = 2$, $2 \mid 30$ so 2 solutions. Reduce: $7x \equiv 15 \pmod{50}$. Inverse of $7 \bmod 50$: $7 \cdot 43 = 301 = 6 \cdot 50 + 1$, so $inv = 43$. $x_0 = 15 \cdot 43 \bmod 50 = 645 \bmod 50 = 45$. Solutions: $x = 45 + t \cdot 50$: $x = 45, 95 \pmod{100}$. Check: $14 \cdot 45 = 630 \equiv 30$; $14 \cdot 95 = 1330 \equiv 30 \pmod{100}$. ✓

Example: (Assignment) $15x \equiv 35 \pmod{50}$. $d = \gcd(15, 50) = 5$, $5 \mid 35$: 5 solutions. Reduce: $3x \equiv 7 \pmod{10}$. $3 \cdot 7 = 21 \equiv 1$, so $inv = 7$. $x_0 = 7 \cdot 7 \bmod 10 = 49 \bmod 10 = 9$. Solutions $x = 9 + 10t$: **9, 19, 29, 39, 49** $\pmod{50}$. Check: $15 \cdot 9 = 135 = 2 \cdot 50 + 35$. ✓

Example: $6x \equiv 5 \pmod{9}$: $d = 3 \nmid 5$, hence no solution.

2.7 Fermat's Little Theorem

Fermat's little theorem

If p is prime and $p \nmid a$ (i.e. $\gcd(a, p) = 1$): $a^{p-1} \equiv 1 \pmod{p}$. Equivalent form valid for *all* integers a : $a^p \equiv a \pmod{p}$.

Example: $p = 7, a = 2$: $2^6 = 64 = 9 \cdot 7 + 1 \equiv 1$. $p = 5, a = 3$: $3^5 = 243 = 48 \cdot 5 + 3 \equiv 3$.

Uses: primality testing, RSA, computing inverses ($a^{-1} \equiv a^{p-2} \pmod{p}$), reducing huge exponents modulo a prime.

Converse is false: Carmichael numbers such as $561 = 3 \cdot 11 \cdot 17$ satisfy $a^{560} \equiv 1 \pmod{561}$ for all a coprime to 561 although composite – so the plain Fermat test is unreliable and Miller–Rabin is used instead.

Example: (Question 2.4) $2^{100} \bmod 13$. 13 prime, $\gcd(2, 13) = 1$, so $2^{12} \equiv 1$. $100 = 12 \cdot 8 + 4$, hence $2^{100} = (2^{12})^8 \cdot 2^4 \equiv 1^8 \cdot 16 = 16 \equiv 3 \pmod{13}$.

Example: (Assignment) $3^{100} \bmod 7$. $3^6 \equiv 1 \pmod{7}$; $100 = 6 \cdot 16 + 4$; $3^{100} \equiv 3^4 = 81 = 11 \cdot 7 + 4 \equiv 4$.

2.8 Miller–Rabin Randomized Primality Test

Miller–Rabin test

A **probabilistic** test: if it answers “composite” it is certainly right; if it answers “probably prime” the error probability is at most 4^{-k} after k independent random bases. **Idea:** if n is an odd prime, $n - 1 = 2^s d$ (d odd), then for any $a \in [2, n - 2]$ either $a^d \equiv 1 \pmod{n}$ or $a^{2^r d} \equiv -1 \pmod{n}$ for some $0 \leq r < s$. If neither holds, a is a **witness** that n is composite.

Algorithm.

1. If $n \leq 3$ return ($n = 2$ or 3); if n even return composite.
2. Write $n - 1 = 2^s \cdot d$ with d odd (divide $n - 1$ by 2 until odd, counting s).

3. Repeat k times: pick random $a \in [2, n-2]$; $x \leftarrow a^d \bmod n$. If $x = 1$ or $x = n-1$, continue to next a . Square x up to $s-1$ times ($x \leftarrow x^2 \bmod n$); if $x = n-1$ stop this round (pass). If $n-1$ was never reached, return **composite**.
4. Return **probably prime**.

```

import random
def miller_rabin(n, k):
    if n <= 3: return n == 2 or n == 3
    if n % 2 == 0: return False
    d, s = n - 1, 0
    while d % 2 == 0:
        d //= 2; s += 1
    for _ in range(k):
        a = random.randint(2, n - 2)
        x = pow(a, d, n)
        if x == 1 or x == n - 1: continue
        for _ in range(s - 1):
            x = pow(x, 2, n)
            if x == n - 1: break
        else:
            return False          # composite
    return True                  # probably prime

```

Choosing k : $n < 100$: 1–3; $100 \leq n \leq 1000$: 5–10; cryptographic sizes: 40 or more. **Complexity:** each round does one modular exponentiation, $O(\log n)$ multiplications, so $O(k \log n)$ multiplications overall (bit cost $O(k \log^3 n)$ with schoolbook arithmetic).

Example: $n = 13, k = 2, a = 4$ and 5 . $n-1 = 12 = 2^2 \cdot 3$: $s = 2, d = 3$. $a = 4$: $x = 4^3 \bmod 13 = 64 \bmod 13 = 12 = n-1$: pass. $a = 5$: $x = 5^3 \bmod 13 = 125 \bmod 13 = 8$, not 1 or 12; inner loop ($s-1 = 1$ time): $x = 8^2 \bmod 13 = 64 \bmod 13 = 12 = n-1$: pass. Both pass: **13 is probably prime**.

Example: (Question 2.5) $n = 17, k = 2, a = 3$ then 5 . $n-1 = 16 = 2^4 \cdot 1$: $s = 4, d = 1$. $a = 3$: $x = 3$; squares: $3^2 = 9$; $9^2 = 81 \bmod 17 = 13$; $13^2 = 169 \bmod 17 = 16 = n-1$ (third square): pass. $a = 5$: $x = 5$; $25 \bmod 17 = 8$; $64 \bmod 17 = 13$; $169 \bmod 17 = 16$: pass. **17 is probably prime** (indeed prime).

Example: (Assignment) $n = 15$. $14 = 2^1 \cdot 7$: $s = 1, d = 7$. Take $a = 2$: $x = 2^7 = 128 \bmod 15 = 8$; neither 1 nor 14; the inner loop runs $s-1 = 0$ times, so return **composite** ($15 = 3 \cdot 5$).

(Assignment) $n = 21$. $20 = 2^2 \cdot 5$: $s = 2, d = 5$. $a = 2$: $x = 2^5 = 32 \bmod 21 = 11$; inner loop once: $11^2 = 121 \bmod 21 = 16 \neq 20$. Composite ($21 = 3 \cdot 7$).

Chapter 2 recap. Euclid: $\gcd(a, b) = \gcd(b, a \bmod b)$, $O(\log \min(a, b))$. Extended Euclid gives $ax + by = \gcd$, hence modular inverses. CRT: $x = \sum a_i M_i y_i \bmod M$. $ax \equiv b \pmod{m}$ has $d = \gcd(a, m)$ solutions iff $d \mid b$. Fermat: $a^{p-1} \equiv 1 \pmod{p}$. Miller–Rabin: $n-1 = 2^s d$, error $\leq 4^{-k}$. Sorting: bubble/insertion $O(n^2)$ best, all three $O(n^2)$ worst, $O(1)$ space.

Chapter 3 Divide and Conquer Algorithms

Divide and conquer

A design paradigm with three steps: **Divide** the problem into a smaller subproblems of the same type (size $\approx n/b$); **Conquer** them recursively (solve directly when small enough, the *base case*); **Combine** the sub-solutions into the solution of the original problem. The running time satisfies $T(n) = aT(n/b) + D(n) + C(n)$, solved by the master theorem of Chapter 1.

3.1 Binary Search

Works on a *sorted* array. Compare the key with the middle element; if equal, done; if smaller, search the left half; otherwise the right half.

```
def binary_search(A, key):
    low, high = 0, len(A) - 1
    while low <= high:
        mid = (low + high) // 2
        if A[mid] == key: return mid
        elif A[mid] < key: low = mid + 1
        else: high = mid - 1
    return -1
```

Example: Search 23 in $A = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]$ (indices 0–9).

Probe	low	high	mid	$A[mid]$	Decision
1	0	9	4	16	$16 < 23$: low = 5
2	5	9	7	56	$56 > 23$: high = 6
3	5	6	5	23	found at index 5

Analysis. Recurrence $T(n) = T(n/2) + 1$ with $T(1) = 1$: master theorem ($a = 1, b = 2, n^{\log_2 1} = 1 = \Theta(f)$, case 2) gives $\Theta(\log n)$. Unrolling: $T(n) = T(n/2^k) + k$; stop at $n/2^k = 1$, so $k = \log_2 n$.

Best case	key at the first middle	$\Theta(1)$
Worst / average	key absent or at a leaf of the decision tree	$\Theta(\log n)$ ($\lfloor \log_2 n \rfloor + 1$ comparisons)
Space	iterative $O(1)$; recursive $O(\log n)$ stack	

3.2 Finding Maximum and Minimum

Straightforward: scan once, comparing with the current max and min: $2(n - 1)$ comparisons.

Divide and conquer: split the array into halves, find (max, min) of each, then compare the two maxima and the two minima (2 comparisons). For one element: both are that element (0 comparisons); for two elements: 1 comparison.

```
def maxmin(A, lo, hi):
    if lo == hi: return A[lo], A[lo]
    if hi == lo + 1: return (A[hi], A[lo]) if A[lo] < A[hi] else (A[lo], A[hi])
    mid = (lo + hi) // 2
    mx1, mn1 = maxmin(A, lo, mid)
    mx2, mn2 = maxmin(A, mid + 1, hi)
    return max(mx1, mx2), min(mn1, mn2)    # 2 comparisons
```

Recurrence: $T(n) = 2T(n/2) + 2$, $T(2) = 1$. For $n = 2^k$:

$$T(n) = 2^{k-1}T(2) + \sum_{i=1}^{k-1} 2^i = 2^{k-1} + (2^k - 2) = \frac{n}{2} + n - 2 = \frac{3n}{2} - 2.$$

So $\frac{3n}{2} - 2$ comparisons versus $2n - 2$ for the naive method: both $\Theta(n)$, but the constant is 25% smaller. Space $O(\log n)$ for recursion.

Example: $A = [8, 3, 9, 1, 7, 5, 6, 2]$: pairs $(8, 3) \rightarrow (8, 3)$; $(9, 1) \rightarrow (9, 1)$; $(7, 5) \rightarrow (7, 5)$; $(6, 2) \rightarrow (6, 2)$ (4 comparisons); merging gives $(9, 1)$ and $(7, 2)$ (4 comparisons) and finally $(9, 1)$ (2 comparisons). Total $4 + 4 + 2 = 10 = \frac{3 \cdot 8}{2} - 2$. ✓

3.3 Merge Sort

Merge sort

Divide the array into two halves; **conquer** by sorting each half recursively; **combine** by *merging* the two sorted halves into one sorted array using two pointers and a temporary array.

```
def merge_sort(A):
    if len(A) <= 1: return A
    mid = len(A) // 2
    L, R = merge_sort(A[:mid]), merge_sort(A[mid:])
    return merge(L, R)

def merge(L, R):
    out, i, j = [], 0, 0
    while i < len(L) and j < len(R):
        if L[i] <= R[j]: out.append(L[i]); i += 1 # <= keeps it stable
        else: out.append(R[j]); j += 1
    return out + L[i:] + R[j:]
```

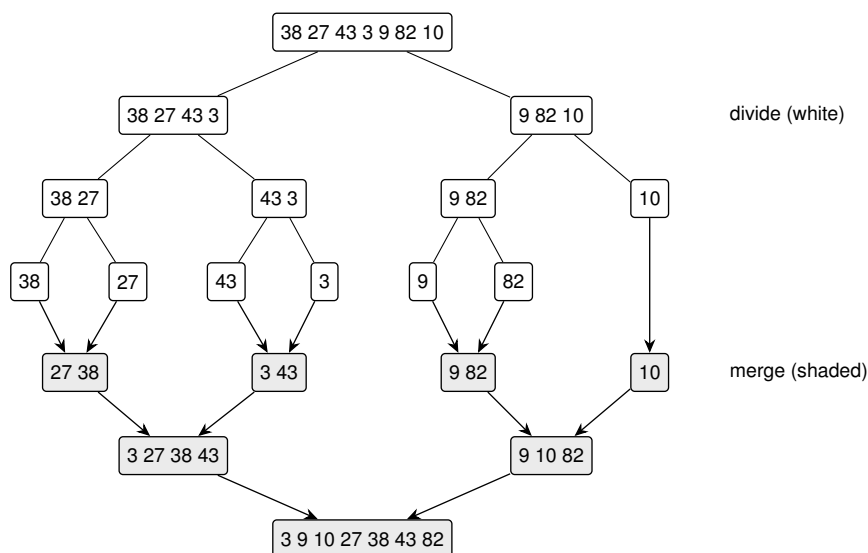


Figure 3: Merge sort on $[38, 27, 43, 3, 9, 82, 10]$: $\lceil \log_2 7 \rceil = 3$ levels of splitting, then 3 levels of merging.

Analysis. Merging two sorted lists with total m elements takes at most $m - 1$ comparisons, i.e. $\Theta(m)$. Recurrence $T(n) = 2T(n/2) + cn$, $T(1) = c'$. Recursion tree: each of the $\log_2 n$ levels costs cn , so

$$T(n) = cn \log_2 n + c'n = \Theta(n \log n) \quad \text{in best, average and worst cases.}$$

(Master theorem case 2.) **Space:** $\Theta(n)$ for the temporary array plus $O(\log n)$ stack. **Stable**, not in-place; ideal for linked lists and external sorting.

3.4 Heap Sort

Binary heap

A **binary heap** is a complete binary tree stored in an array. With 1-based indexing, node i has parent $\lfloor i/2 \rfloor$, left child $2i$, right child $2i + 1$. **Max-heap property:** $A[\text{parent}(i)] \geq A[i]$ (root is the maximum). Height = $\lfloor \log_2 n \rfloor$.

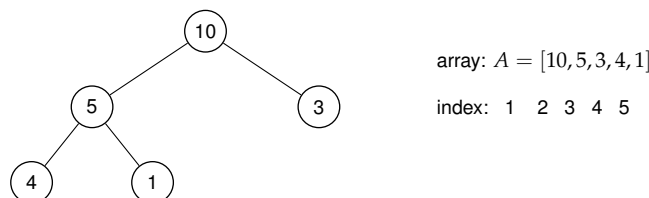


Figure 4: A max-heap and its array form.

MAX-HEAPIFY(A, i): assuming the subtrees of i are max-heaps, let $\ell = 2i$, $r = 2i + 1$; find the largest among $A[i]$, $A[\ell]$, $A[r]$; if it is not $A[i]$, swap and recurse on that child. Cost $O(h) = O(\log n)$ (recurrence $T(n) \leq T(2n/3) + \Theta(1)$, since a child subtree has at most $2n/3$ nodes; case 2 gives $O(\log n)$).

BUILD-MAX-HEAP: for $i = \lfloor n/2 \rfloor$ down to 1: **MAX-HEAPIFY**(A, i). A naive bound is $O(n \log n)$, but a node at height h costs $O(h)$ and there are at most $\lceil n/2^{h+1} \rceil$ such nodes:

$$\sum_{h=0}^{\lfloor \log n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h}\right) = O(2n) = O(n),$$

using $\sum_{h \geq 0} h/2^h = 2$.

HEAPSORT: **BUILD-MAX-HEAP**; for $i = n$ down to 2: swap $A[1] \leftrightarrow A[i]$, decrement heap size, **MAX-HEAPIFY**($A, 1$).

```
def max_heapify(A, n, i):
    # 0-based; heap occupies A[0..n-1]
    l, r, largest = 2*i + 1, 2*i + 2, i
    if l < n and A[l] > A[largest]: largest = l
    if r < n and A[r] > A[largest]: largest = r
    if largest != i:
        A[i], A[largest] = A[largest], A[i]
        max_heapify(A, n, largest)

def heap_sort(A):
    n = len(A)
    for i in range(n // 2 - 1, -1, -1): max_heapify(A, n, i) # build heap
    for end in range(n - 1, 0, -1):
        A[0], A[end] = A[end], A[0]
        max_heapify(A, end, 0)
    return A
```

Example: **Trace**, $A = [4, 10, 3, 5, 1]$.

Build heap	$i = 2$: $10 \geq 5, 1$ ok. $i = 1$: $4 < 10$, swap $\rightarrow [10, 4, 3, 5, 1]$; node 4 vs children 5, 1: swap $\rightarrow [10, 5, 3, 4, 1]$.
Extract 1	swap root with last: $[1, 5, 3, 4 10]$; heapify: $1 \leftrightarrow 5$, then $1 \leftrightarrow 4 \rightarrow [5, 4, 3, 1 10]$.
Extract 2	$[1, 4, 3 5, 10]$; heapify: $1 \leftrightarrow 4 \rightarrow [4, 1, 3 5, 10]$.
Extract 3	$[3, 1 4, 5, 10]$; root $3 \geq 1$ ok.
Extract 4	$[1 3, 4, 5, 10]$. Sorted: $[1, 3, 4, 5, 10]$.

Analysis. Build: $O(n)$; $n - 1$ extractions each $O(\log n)$: $T(n) = O(n) + (n - 1)O(\log n) = \Theta(n \log n)$ in all cases. Space $O(1)$ (in-place); *not stable*. Also the structure behind priority queues.

3.5 Quick Sort

Quick sort

Divide: choose a **pivot** and **partition** so that all elements $\leq$ pivot are to its left and all $>$ pivot to its right (the pivot is now in its final place). **Conquer:** sort the two parts recursively. **Combine:** nothing (sorting happens in place).

```
def partition(A, lo, hi):          # Lomuto scheme, pivot = A[hi]
    pivot, i = A[hi], lo - 1
    for j in range(lo, hi):
        if A[j] <= pivot:
            i += 1
            A[i], A[j] = A[j], A[i]
    A[i + 1], A[hi] = A[hi], A[i + 1]
    return i + 1

def quick_sort(A, lo, hi):
    if lo < hi:
        q = partition(A, lo, hi)
        quick_sort(A, lo, q - 1)
        quick_sort(A, q + 1, hi)
```

Example: **Partition of** $A = [2, 8, 7, 1, 3, 5, 6, 4]$, **pivot** 4.

j	$A[j]$	i	Array after step
0	$2 (\leq 4)$	0	$[2, 8, 7, 1, 3, 5, 6, 4]$
1	8	0	unchanged
2	7	0	unchanged
3	$1 (\leq 4)$	1	swap $A[1], A[3]$: $[2, 1, 7, 8, 3, 5, 6, 4]$
4	$3 (\leq 4)$	2	swap $A[2], A[4]$: $[2, 1, 3, 8, 7, 5, 6, 4]$
5, 6	5, 6	2	unchanged
end	pivot	–	swap $A[3]$ with pivot: $[2, 1, 3, 4, 7, 5, 6, 8], q = 3$

Partition costs $\Theta(n)$ ($n - 1$ comparisons). The recurrence is $T(n) = T(q) + T(n - q - 1) + \Theta(n)$.

- **Best case** (pivot is always the median): $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$.
- **Worst case** (pivot is always the smallest or largest: already sorted/reverse sorted input with first/last pivot): $T(n) = T(n - 1) + \Theta(n)$, so $T(n) = \sum_{k=1}^n ck = \Theta(n^2)$.
- **Average case.** For an even split 9:1 at every level, $T(n) = T(9n/10) + T(n/10) + cn$ is still $O(n \log n)$ (tree depth $\log_{10/9} n$, cost $\leq cn$ per level). Exactly, with all n pivot positions equally likely,

$$T(n) = \frac{1}{n} \sum_{q=0}^{n-1} [T(q) + T(n - 1 - q)] + cn = \frac{2}{n} \sum_{q=0}^{n-1} T(q) + cn \Rightarrow T(n) \approx 2n \ln n \approx 1.39 n \log_2 n.$$

Space $O(\log n)$ expected stack (in-place); not stable.

Randomized quick sort

Randomized quick sort

Choose the pivot *uniformly at random* from $A[lo..hi]$ (swap it into position hi , then partition as before). No fixed input is bad: the worst case is still $\Theta(n^2)$ but only with tiny probability, and the **expected** running time is $O(n \log n)$ for every input.

Expected-time proof. Let $z_1 < \dots < z_n$ be the sorted elements and $X_{ij} = 1$ if z_i is compared with z_j (at most once, since a pivot is excluded afterwards). Elements z_i, z_j are compared iff the first pivot chosen

among $z_i, \dots, z_j$ is z_i or z_j , so $\Pr[X_{ij} = 1] = \frac{2}{j-i+1}$. By linearity of expectation

$$E[\text{\#comparisons}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} < \sum_{i=1}^{n-1} O(\log n) = O(n \log n).$$

Algorithm	Best	Average	Worst	Space	Stable
Merge sort	$n \log n$	$n \log n$	$n \log n$	$O(n)$	yes
Heap sort	$n \log n$	$n \log n$	$n \log n$	$O(1)$	no
Quick sort	$n \log n$	$n \log n$	n^2	$O(\log n)$	no
Randomized quick sort	$n \log n$	$n \log n$ (expected)	n^2 (rare)	$O(\log n)$	no

Lower bound. Any comparison sort is a binary decision tree with at least $n!$ leaves, so its height is $\geq \log_2 n! = \Theta(n \log n)$. Merge sort and heap sort are therefore asymptotically optimal.

3.6 Order Statistics

The i -th **order statistic** of a set of n elements is its i -th smallest element (minimum: $i = 1$; maximum: $i = n$; median: $i = \lceil n/2 \rceil$). Sorting then indexing costs $O(n \log n)$; the algorithms below need only $O(n)$.

Selection in expected linear time (randomized select)

Partition around a random pivot; if the pivot's rank k equals i , return it; else recurse into *only one* side.

```
import random
def rselect(A, lo, hi, i):          # i-th smallest (1-based) in A[lo..hi]
    if lo == hi: return A[lo]
    r = random.randint(lo, hi); A[r], A[hi] = A[hi], A[r]
    q = partition(A, lo, hi)
    k = q - lo + 1                  # rank of the pivot
    if i == k: return A[q]
    elif i < k: return rselect(A, lo, q - 1, i)
    else: return rselect(A, q + 1, hi, i - k)
```

Example: 3rd smallest of $[7, 10, 4, 3, 20, 15]$ (pivot 15): partition $\rightarrow [7, 10, 4, 3, 15, 20]$, rank $5 > 3$, recurse left $[7, 10, 4, 3]$ (pivot 3): $\rightarrow [3, 10, 4, 7]$, rank $1 < 3$, search $[10, 4, 7]$ for $i = 2$ (pivot 7): $\rightarrow [4, 7, 10]$, rank 2: answer 7. ✓

Analysis. Worst case $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$ (bad pivots). Expectation: the larger side is where we recurse, and each rank $k \in \{0..n-1\}$ is equally likely, so

$$E[T(n)] \leq \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[T(k)] + an.$$

Substitute the guess $E[T(k)] \leq ck$: $\sum_{k=\lfloor n/2 \rfloor}^{n-1} k \leq \frac{3n^2}{8}$ (up to lower-order terms), giving $E[T(n)] \leq \frac{3cn}{4} + \Theta(c) + an = cn - (\frac{cn}{4} - \Theta(c) - an) \leq cn$ for c large enough. Hence $E[T(n)] = O(n)$.

Selection in worst-case linear time (median of medians)

SELECT (Blum–Floyd–Pratt–Rivest–Tarjan)

1. Divide the n elements into $\lceil n/5 \rceil$ groups of 5 (the last may be smaller).
2. Sort each group (insertion sort, $O(1)$ per group) and take its median: $\lceil n/5 \rceil$ medians.
3. Recursively find the median x of these medians (the “median of medians”).
4. Partition the whole array around x ; let k be its rank.
5. If $i = k$ return x ; if $i < k$ recurse on the low side; else recurse on the high side with $i - k$.

Why x is a good pivot. At least half of the $\lceil n/5 \rceil$ medians are $\geq x$; each such group has 3 elements $\geq x$ (except possibly the group of x and the incomplete group). So the number of elements $\geq x$ is at least $3(\lceil \frac{1}{2} \lceil \frac{n}{5} \rceil \rceil - 2) \geq \frac{3n}{10} - 6$. Symmetrically at least $\frac{3n}{10} - 6$ elements are $\leq x$. Therefore the recursive call in step 5 is on at most $n - (\frac{3n}{10} - 6) = \frac{7n}{10} + 6$ elements.

Recurrence: steps 1, 2, 4 cost $O(n)$; step 3 costs $T(\lceil n/5 \rceil)$; step 5 costs at most $T(7n/10 + 6)$:

$$T(n) \leq T(\lceil n/5 \rceil) + T(7n/10 + 6) + an \quad (n \geq n_0).$$

Substitution: guess $T(n) \leq cn$:

$$T(n) \leq c\left(\frac{n}{5} + 1\right) + c\left(\frac{7n}{10} + 6\right) + an = \frac{9cn}{10} + 7c + an = cn + \left(7c + an - \frac{cn}{10}\right) \leq cn$$

whenever $\frac{cn}{10} - 7c \geq an$, i.e. $c \geq \frac{10an}{n-70}$, which holds for $n > 70$ (e.g. $c = 20a$ for $n \geq 140$) – note that $\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1$ is the key. Hence $T(n) = O(n)$ in the worst case. (Group size 3 would give $\frac{1}{3} + \frac{2}{3} = 1$ and fail.) The constant is large, so randomized select is faster in practice.

Chapter 3 recap. Binary search $\Theta(\log n)$; max–min $\frac{3n}{2} - 2$; merge sort always $\Theta(n \log n)$ with $O(n)$ extra space; heap sort $\Theta(n \log n)$ in place (build-heap $O(n)$); quick sort $O(n \log n)$ average, $O(n^2)$ worst, randomization makes the bad case unlikely; selection is $O(n)$ expected (random pivot) and $O(n)$ worst case (median of medians, $T(n) \leq T(n/5) + T(7n/10 + 6) + O(n)$).

Chapter 4 Greedy Algorithms

4.1 Basic Concepts

Optimization problem

A problem with a set of **feasible solutions** (satisfying the constraints), each having a cost or value; the goal is to *minimize* (shortest path, minimum spanning tree) or *maximize* (profit, knapsack) it. The best feasible solution is the **optimal solution**; the best overall is the **global optimum**, the best in a small neighbourhood a **local optimum**.

Greedy algorithm

Builds the solution step by step, at each step making the choice that looks best *right now* (locally optimal) without considering future consequences. Each choice must be **feasible**, **locally optimal** and **irrevocable** (never undone).

Elements of the greedy strategy: (1) **candidate set** – items from which a choice is made; (2) **selection function** – picks the best candidate by a criterion (the heuristic); (3) **feasibility function** – checks that adding the candidate violates no constraint; (4) **objective function** – value to maximize/minimize; (5) **solution function** – tells whether a complete solution has been reached.

When does greedy give the optimum? Two properties are required: the **greedy-choice property** (a globally optimal solution can be built by making locally optimal choices) and **optimal substructure** (an optimal solution contains optimal solutions to its subproblems). Proofs are usually by an *exchange argument*: take any optimal solution and swap its first choice for the greedy choice without making it worse.

Generic template

```
def greedy(C):
    # C = candidate set
    S = [] # solution
    while C and not solved(S):
        x = select(C) # selection function (the "greedy" rule)
        C.remove(x)
        if feasible(S + [x]): # feasibility function
            S.append(x)
    return S
```

Greedy can fail: 0/1 knapsack, items (weight, value) = (10, 60), (20, 100), (30, 120), capacity 50. Best ratio first (6, 5, 4): take items 1 and 2: value 160, weight 30; item 3 does not fit. The optimum is items 2 and 3: 220. Hence 0/1 knapsack needs dynamic programming (Chapter 5), while the *fractional* version (next section) is solved optimally by greedy: $160 + \frac{20}{30} \cdot 120 = 240$.

4.2 Fractional Knapsack Problem

Fractional knapsack

Given n items with weights w_i and values v_i and capacity M , choose fractions $x_i \in [0, 1]$ maximizing $\sum v_i x_i$ subject to $\sum w_i x_i \leq M$. (In the 0/1 variant $x_i \in \{0, 1\}$.)

Greedy rule: sort items by decreasing **value density** $p_i = v_i/w_i$; take whole items while they fit; take a fraction of the next item to fill the knapsack exactly.

Algorithm: GREEDY_FRACTIONAL_KNAPSACK(X, V, W, M)

Precondition: items sorted by v_i/w_i decreasing. $SW \leftarrow 0, SP \leftarrow 0$. For $i = 1$ to n : if $SW + W[i] \leq M$: take all of item i ($SW \leftarrow SW + W[i], SP \leftarrow SP + V[i]$); else $frac \leftarrow (M - SW)/W[i]$, take that fraction ($SP \leftarrow SP + V[i] \cdot frac, SW \leftarrow M$) and stop. Return (S, SW, SP).

Complexity. Sorting $O(n \log n)$; one pass $O(n)$; so $T(n) = O(n \log n)$. Extra space $S(n) = O(1)$ (a few variables).

Correctness (exchange argument). Suppose an optimal solution leaves some weight of a denser item i unused while holding weight of a less dense item j ($p_i > p_j$). Replacing a small weight δ of j by δ of i changes the value by $\delta(p_i - p_j) > 0$, contradicting optimality. So the optimum fills items in density order.

Example: $n = 3, M = 20, V = (24, 25, 15), W = (18, 15, 20)$. Densities: $I_1 : 24/18 = 1.333, I_2 : 25/15 = 1.667, I_3 : 15/20 = 0.75$. Order: I_2, I_1, I_3 .

- $i = 1$ (I_2): $0 + 15 \leq 20$: take all. $SW = 15, SP = 25$.
- $i = 2$ (I_1): $15 + 18 = 33 > 20$: $frac = (20 - 15)/18 = 0.2778$. $SP = 25 + 24(0.2778) = 31.67$, $SW = 15 + 18(0.2778) = 20$. Knapsack full.

Solution: $S = \{I_2, 0.2778 I_1\}$, profit = 31.67, weight = 20.

Example: (Question 4.1) $M = 28$, profits $P = (9, 5, 2, 7, 6, 16, 3)$, weights $W = (2, 5, 6, 11, 1, 9, 1)$. Densities: I_1 4.50, I_2 1.00, I_3 0.33, I_4 0.636, I_5 6.00, I_6 1.778, I_7 3.00. Order: $I_5, I_1, I_7, I_6, I_2, I_4, I_3$.

Item taken	Weight added	SW	Profit added	SP	Note
I_5	1	1	6	6	whole
I_1	2	3	9	15	whole
I_7	1	4	3	18	whole
I_6	9	13	16	34	whole
I_2	5	18	5	39	whole
I_4	10 (of 11)	28	$7 \cdot \frac{10}{11} = 6.36$	45.36	fraction 0.909

Answer: $S = \{I_5, I_1, I_7, I_6, I_2, 0.91 I_4\}$, $SW = 28, SP \approx 45.36$ (the lecture key rounds to 45.37 because it uses the rounded fraction 0.91).

4.3 Job Sequencing with Deadlines

Job sequencing with deadlines

N jobs, each taking *unit time* on one processor, with deadline $d_i \geq 1$ and profit $p_i \geq 0$ earned only if job i finishes at or before d_i . Choose and order a subset of jobs maximizing total profit. A set is **feasible** if its jobs can be scheduled so every one meets its deadline. Slot t spans time $(t - 1, t]$.

Greedy rule: consider jobs in decreasing order of profit; place each job in the *latest free slot* $\leq d_i$; if no such slot exists, discard it.

Algorithm: JOB_SCHEDULING(J, D, P)

Sort jobs by profit decreasing. $S \leftarrow \emptyset, SP \leftarrow 0$, all slots $1.. \max d$ empty. For each job i in that order: if a free slot exists in $1..d_i$, put the job in the latest such slot, $S \leftarrow S \cup \{i\}, SP \leftarrow SP + p_i$. Return (S, SP).

Complexity. Sorting $O(n \log n)$; for each job scanning slots from d_i down costs up to $O(n)$, so $T(n) = O(n^2)$; with a disjoint-set (union–find) structure to find the latest free slot it falls to nearly $O(n \log n)$. Space $O(n)$ for the slot array and set S .

Correctness: the feasible sets form a matroid, for which greedy-by-weight is optimal.

Example: $N = 4$, $P = (100, 10, 15, 27)$, $D = (2, 1, 2, 1)$. Sorted by profit: $J_1(100, d=2)$, $J_4(27, 1)$, $J_3(15, 2)$, $J_2(10, 1)$.

- J_1 : latest free slot ≤ 2 is slot 2. $S = \{J_1\}$, $SP = 100$.
- J_4 : slot 1 free. Place in slot 1. $SP = 127$.
- J_3 ($d = 2$): slots 2 and 1 occupied: discard. J_2 ($d = 1$): slot 1 occupied: discard.

Schedule: slot 1 = J_4 , slot 2 = J_1 ; profit = 127.

Example: $N = 7$; $(J_1:4, 70)$, $(J_2:2, 60)$, $(J_3:1, 30)$, $(J_4:4, 50)$, $(J_5:6, 10)$, $(J_6:3, 40)$, $(J_7:4, 20)$ (deadline, profit). Sorted: $J_1(70, 4)$, $J_2(60, 2)$, $J_4(50, 4)$, $J_6(40, 3)$, $J_3(30, 1)$, $J_7(20, 4)$, $J_5(10, 6)$.

Job	Deadline	Slot chosen	Occupied slots after
J_1	4	4	{4}
J_2	2	2	{2, 4}
J_4	4	3 (4 taken)	{2, 3, 4}
J_6	3	1 (3, 2 taken)	{1, 2, 3, 4}
J_3	1	none (slot 1 taken): discard	–
J_7	4	none (1–4 full): discard	–
J_5	6	6 (latest free ≤ 6)	{1, 2, 3, 4, 6}

Schedule by slot: $J_6, J_2, J_4, J_1, -, J_5$. Profit = $70 + 60 + 50 + 40 + 10 = 230$.

Example: (Question 4.2) $n = 7$, $P = (3, 5, 20, 18, 1, 6, 30)$, $D = (1, 3, 4, 3, 2, 1, 2)$. Sorted: $J_7(30, 2)$, $J_3(20, 4)$, $J_4(18, 3)$, $J_6(6, 1)$, $J_2(5, 3)$, $J_1(3, 1)$, $J_5(1, 2)$. $J_7 \rightarrow$ slot 2; $J_3 \rightarrow$ slot 4; $J_4 \rightarrow$ slot 3; $J_6 \rightarrow$ slot 1; J_2 ($d = 3$): slots 3, 2, 1 taken, discard; J_1, J_5 discard. Selected $\{J_6, J_7, J_4, J_3\}$ in slot order; profit $30 + 20 + 18 + 6 = 74$.

4.4 Minimum Spanning Tree Algorithms

Minimum spanning tree (MST)

For a connected, weighted, undirected graph $G = (V, E)$ a **spanning tree** is an acyclic connected subgraph containing all $|V|$ vertices and exactly $|V| - 1$ edges. An MST is a spanning tree of minimum total weight. Both Kruskal's and Prim's algorithms are greedy and correct because of the **cut property**: for any cut of V , a minimum-weight edge crossing the cut belongs to some MST.

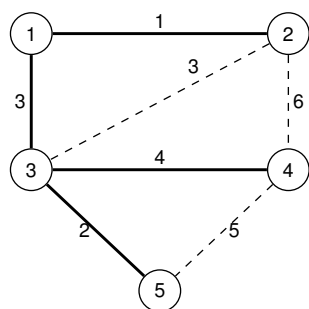
Kruskal's algorithm (edge-based)

Principle: sort all edges by weight; scan them in increasing order; add an edge if it joins two different components (creates no cycle); stop after $|V| - 1$ edges. Cycle check with **union–find** (disjoint sets): $\text{find}(u) \neq \text{find}(v)$, then $\text{union}(u, v)$.

Algorithm: KRUSKAL_MST(G)

Sort E in non-decreasing order of weight. $MST \leftarrow \emptyset$. While $|MST| < |V| - 1$ and edges remain: take the lightest remaining edge e ; if adding e to MST forms no cycle, $MST \leftarrow MST \cup \{e\}$; discard e . Return MST .

Complexity. Sorting $O(|E| \log |E|)$; union–find on $|E|$ edges $O(|E| \alpha(|V|)) \approx O(|E|)$. Since $|E| \leq |V|^2$, $\log |E| \leq 2 \log |V|$, so $T = O(|E| \log |V|)$. Space $S = O(|V| + |E|)$. Good for sparse graphs.



bold: MST edges, total = $1 + 2 + 3 + 4 = 10$

dashed: rejected (create a cycle)

Figure 5: Graph of the Kruskal example and its MST.

Example: **Vertices** $\{1..5\}$; **edges** $\langle 1,2 \rangle:1, \langle 1,3 \rangle:3, \langle 2,3 \rangle:3, \langle 2,4 \rangle:6, \langle 3,4 \rangle:4, \langle 3,5 \rangle:2, \langle 4,5 \rangle:5$. **Sorted:** $\langle 1,2 \rangle 1, \langle 3,5 \rangle 2, \langle 1,3 \rangle 3, \langle 2,3 \rangle 3, \langle 3,4 \rangle 4, \langle 4,5 \rangle 5, \langle 2,4 \rangle 6$.

Step	Edge (wt)	Components before	Decision
1	$\langle 1,2 \rangle (1)$	$\{1\}, \{2\}$	add
2	$\langle 3,5 \rangle (2)$	$\{3\}, \{5\}$	add
3	$\langle 1,3 \rangle (3)$	$\{1,2\}, \{3,5\}$	add $\rightarrow \{1,2,3,5\}$
4	$\langle 2,3 \rangle (3)$	same component	reject (cycle)
5	$\langle 3,4 \rangle (4)$	$\{1,2,3,5\}, \{4\}$	add; $ MST = 4 = V - 1$: stop

$MST = \{\langle 1,2 \rangle, \langle 3,5 \rangle, \langle 1,3 \rangle, \langle 3,4 \rangle\}$, cost = $1 + 2 + 3 + 4 = 10$.

Example: (Question 4.3) Seven-vertex graph. Edges: $v_1v_7:1, v_3v_4:3, v_2v_7:4, v_4v_5:7, v_3v_7:9, v_2v_3:15, v_4v_7:16, v_1v_2:20, v_1v_6:23, v_5v_7:25, v_5v_6:28, v_6v_7:36$ (sorted). Kruskal: add v_1v_7 (1); add v_3v_4 (3); add v_2v_7 (4); add v_4v_5 (7) $\rightarrow \{3,4,5\}$; add v_3v_7 (9) merges $\{1,2,7\}$ with $\{3,4,5\}$; reject v_2v_3 (15), v_4v_7 (16), v_1v_2 (20) – all within one component; add v_1v_6 (23) brings in v_6 . Six edges for seven vertices: stop. Cost = $1 + 3 + 4 + 7 + 9 + 23 = 47$.

Prim's algorithm (vertex-based)

Principle: start from any vertex; repeatedly add the cheapest edge connecting a tree vertex to a non-tree vertex, until all vertices are in the tree. Preferred for dense graphs.

Algorithm: PRIM(G, s)

$MST \leftarrow \{s\}, UV \leftarrow V \setminus \{s\}$. While $UV \neq \emptyset$: let $NE = \{(u, v) \in E : u \in MST, v \in UV\}$; choose $e = (u, v) \in NE$ of minimum weight; add e to the tree; $MST \leftarrow MST \cup \{v\}; UV \leftarrow UV \setminus \{v\}$. Return the edges and total weight.

Complexity depends on the data structures: the loop runs $n - 1$ times.

Implementation	Find-min / update	Total time	Space
Naive: rescan all m edges each time	$O(m)$ per iteration	$O(n \cdot m), O(n^3)$ if dense	$O(n + m)$
Adjacency matrix + linear search of $key[]$	$O(n)$	$O(n^2)$	$O(n^2)$
Adjacency list + binary heap	$O(\log n)$	$O((n + m) \log n)$	$O(n + m)$
Adjacency list + Fibonacci heap	$O(\log n) / O(1)$ amortized	$O(m + n \log n)$	$O(n + m)$

Kruskal vs Prim: Kruskal $O(|E| \log |V|)$ is better for sparse graphs ($|E| \approx |V|$); Prim with a matrix $O(|V|^2)$ (or Fibonacci heap) is better for dense graphs ($|E| \approx |V|^2$).

Example: **Prim** on the same graph as the Kruskal example, start at vertex 1.

Step	Candidate edges NE (weight)	Chosen	MST vertices
1	$\langle 1,2 \rangle 1, \langle 1,3 \rangle 3$	$\langle 1,2 \rangle$	$\{1,2\}$
2	$\langle 1,3 \rangle 3, \langle 2,3 \rangle 3, \langle 2,4 \rangle 6$	$\langle 1,3 \rangle$ (tie)	$\{1,2,3\}$
3	$\langle 2,4 \rangle 6, \langle 3,4 \rangle 4, \langle 3,5 \rangle 2$	$\langle 3,5 \rangle$	$\{1,2,3,5\}$
4	$\langle 2,4 \rangle 6, \langle 3,4 \rangle 4, \langle 4,5 \rangle 5$	$\langle 3,4 \rangle$	all 5

Cost = $1 + 3 + 2 + 4 = 10$, same as Kruskal.

Example: Lecture example 4.3.2: vertices $a..f$ with edges $ab3, af5, ae6, bc1, bf4, cf4, cd6, fd5, fe2, de8$. Start at a : take $ab(3)$; then $bc(1)$; then $\{af5, ae6, bf4, cf4, cd6\} \rightarrow$ tie bf/cf weight 4, take $cf(4)$; then $fe(2)$; then $fd(5)$ (bf, af now create cycles). MST: ab, bc, cf, fe, fd ; cost $3 + 1 + 4 + 2 + 5 = 15$.

Example: (Question 4.4) Graph 1-2:1, 1-3:3, 2-3:3, 2-4:6, 3-4:4, 3-5:2, 4-5:5 is the same as the Kruskal example: Prim's cost = 10 (shown above).

4.5 Single-Source Shortest Paths: Dijkstra's Algorithm

Single-source shortest path

Given a weighted graph $G = (V, E, w)$ with **non-negative** weights and a source s , find the minimum-weight path from s to every other vertex. Dijkstra's algorithm is greedy: always finalise the unvisited vertex with the smallest tentative distance.

Relaxation: for an edge (u, v) : if $dist[u] + w(u, v) < dist[v]$ then $dist[v] \leftarrow dist[u] + w(u, v)$ and $\pi[v] \leftarrow u$ (π = parent pointer).

Algorithm: DIJKSTRA(G, s)

$dist[s] \leftarrow 0; dist[v] \leftarrow \infty, \pi[v] \leftarrow \text{NIL}$ for all $v \neq s$; put all vertices in a queue Q . While $Q \neq \emptyset$: $u \leftarrow$ vertex in Q with minimum $dist[u]$; remove u ; for each neighbour v of u : relax (u, v) . Return $dist, \pi$.

Correctness sketch: when u is removed, $dist[u]$ is final: any other path to u must leave the finalised set through some vertex whose tentative distance is already $\geq dist[u]$, and since weights are ≥ 0 the rest of that path cannot make it shorter. A *negative* edge breaks this argument, so Dijkstra fails with negative weights (use Bellman–Ford).

Priority-queue implementation	Total time
Array / linear search ($ V $ extract-mins of $O(V)$ each, plus $ V $ init)	$O(V ^2)$
Binary heap	$O((V + E) \log V)$
Fibonacci heap	$O(E + V \log V)$

Space: $dist[], \pi[]$ and Q are each $O(|V|)$, so $S = O(|V|)$ auxiliary ($O(|V| + |E|)$ with the graph).

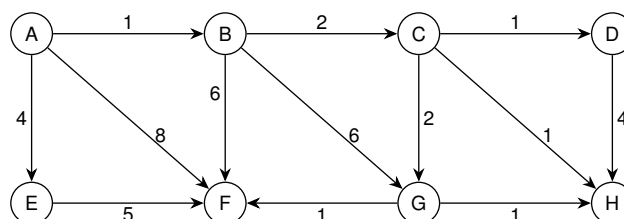


Figure 6: Directed graph for the Dijkstra example, source A .

Example: Dijkstra from A . Edges: $A \rightarrow B1, A \rightarrow E4, A \rightarrow F8, B \rightarrow C2, B \rightarrow F6, B \rightarrow G6, C \rightarrow D1, C \rightarrow G2, C \rightarrow H1, D \rightarrow H4, E \rightarrow F5, G \rightarrow F1, G \rightarrow H1$. Table of $dist$ (parent in brackets) after each vertex is finalised:

It.	u	A	B	C	D	E	F	G	H
init	–	0	∞	∞	∞	∞	∞	∞	∞
1	A	0	1(A)	∞	∞	4(A)	8(A)	∞	∞
2	B	0	1	3(B)	∞	4	7(B)	7(B)	∞
3	C	0	1	3	4(C)	4	7	5(C)	4(C)
4	D (4)	0	1	3	4	4	7	5	4 (4 + 4 = 8 no)
5	E (4)	0	1	3	4	4	7 (4 + 5 = 9 no)	5	4
6	H (4)	–	–	–	–	–	–	–	no out-edges
7	G (5)	0	1	3	4	4	6(G)	5	4 (5 + 1 = 6 no)
8	F (6)	no out-edges							
Final		0	1	3	4	4	6	5	4
Parent		NIL	A	B	C	A	G	C	C

Shortest-path tree edges: $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow D$, $A \rightarrow E$, $C \rightarrow G$, $G \rightarrow F$, $C \rightarrow H$. For instance the path to F is $A \rightarrow B \rightarrow C \rightarrow G \rightarrow F$ with cost $1 + 2 + 2 + 1 = 6$.

Remark. The lecture trace lists the neighbours of C as $\{D, G\}$ only and so reports $\text{dist}[H] = 6$ via G ; the figure, however, shows an edge $C \rightarrow H$ of weight 1, which gives $\text{dist}[H] = 3 + 1 = 4$ as above. If your graph has no $C \rightarrow H$ edge, $H = 6$ via G and every other value is unchanged.

Example: (Assignment) undirected graph: AB_4 , AC_5 , BC_{11} , BD_9 , BE_7 , CE_3 , DE_{13} , DF_2 , EF_6 ; **source** A .

It.	u	A	B	C	D	E	F
init	–	0	∞	∞	∞	∞	∞
1	A	0	4(A)	5(A)	∞	∞	∞
2	B (4)	0	4	5	13(B)	11(B)	∞
3	C (5)	0	4	5	13	8(C)	∞
4	E (8)	0	4	5	13	8	14(E)
5	D (13)	0	4	5	13	8	14 (13 + 2 = 15 no)
6	F (14)	done					
Final		0	4	5	13	8	14
Parent		–	A	A	B	C	E

Shortest-path tree: $A-B$, $A-C$, $B-D$, $C-E$, $E-F$.

Chapter 4 recap. Greedy = locally best, irrevocable choices; needs greedy-choice property + optimal substructure. Fractional knapsack by density $O(n \log n)$; job sequencing by profit, latest free slot $O(n^2)$; Kruskal $O(E \log V)$ (sort edges + union-find); Prim $O(V^2)$ or $O(E \log V)$; Dijkstra $O(V^2)$ / $O((V + E) \log V)$, non-negative weights only.

Chapter 5 Dynamic Programming

5.1 Basic Concepts

Dynamic programming (DP)

A technique for optimization problems whose solutions are built from solutions of **overlapping subproblems**. Each subproblem is solved *once*, its answer is stored in a table, and later reused, turning exponential recursion into polynomial time. Two properties are needed: **optimal substructure** (an optimal solution contains optimal solutions of its subproblems – Bellman’s *principle of optimality*) and **overlapping subproblems** (the same subproblem arises many times).

Two implementations. Top-down with memoization: write the recursion, but check a table before computing and store each result. **Bottom-up tabulation:** fill a table from the smallest subproblems upward in a suitable order.

Recipe: (1) characterize the structure of an optimal solution; (2) write a recurrence for the optimal value; (3) compute values bottom-up in a table; (4) reconstruct the solution by tracing back through the table.

Example: Fibonacci. Naive recursion recomputes $F(k)$ many times: $T(n) = T(n-1) + T(n-2) + 1 = \Theta(\phi^n)$. With a table $F[0..n]$ each value is computed once: $T(n) = \Theta(n)$ time, $\Theta(n)$ space.

	Divide & conquer	Greedy	Dynamic programming
Subproblems	independent	one subproblem after a choice	overlapping
Choice	after solving subproblems	before, irrevocable	after solving, tries all
Optimality	depends on algorithm	only if greedy property holds	always (if substructure holds)
Typical cost	$n \log n$	$n \log n$	polynomial in table size

5.2 All-Pairs Shortest Paths: Floyd–Warshall

Problem: for a weighted directed graph with n vertices (negative edges allowed, no negative cycles) find the shortest distance between *every* pair.

Idea: let $D^{(k)}[i][j]$ be the shortest $i \rightarrow j$ path using only vertices $\{1..k\}$ as intermediates. The path either avoids vertex k or passes through it once:

$$D^{(k)}[i][j] = \min \left(D^{(k-1)}[i][j], D^{(k-1)}[i][k] + D^{(k-1)}[k][j] \right), \quad D^{(0)} = \text{the cost matrix.}$$

```
INF = float('inf')
def floyd_warshall(D):
    # D: n x n cost matrix, D[i][i] = 0
    n = len(D)
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if D[i][k] + D[k][j] < D[i][j]:
                    D[i][j] = D[i][k] + D[k][j]
    return D
```

Complexity. Three nested loops of n iterations: $T(n) = \Theta(n^3)$. The update can be done in place, so space $S(n) = \Theta(n^2)$ (just the matrix). Running Dijkstra from every source gives the same $O(n^3)$ on dense graphs but cannot handle negative edges.

Example: Four-vertex graph. $D^{(0)}$ (a missing edge is ∞):

$$D^{(0)} = \begin{pmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & \infty \\ 5 & \infty & 0 & 1 \\ 2 & \infty & \infty & 0 \end{pmatrix}.$$

$k = 1$ (via vertex 1): $d_{24} = \min(\infty, 8 + 7) = 15$; $d_{32} = \min(\infty, 5 + 3) = 8$; $d_{42} = \min(\infty, 2 + 3) = 5$.

$$D^{(1)} = \begin{pmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & 15 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & \infty & 0 \end{pmatrix}.$$

$k = 2$: $d_{13} = \min(\infty, 3 + 2) = 5$; $d_{43} = \min(\infty, 5 + 2) = 7$; others unchanged.

$$D^{(2)} = \begin{pmatrix} 0 & 3 & 5 & 7 \\ 8 & 0 & 2 & 15 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{pmatrix}.$$

$k = 3$: $d_{14} = \min(7, 5 + 1) = 6$; $d_{21} = \min(8, 2 + 5) = 7$; $d_{24} = \min(15, 2 + 1) = 3$.

$$D^{(3)} = \begin{pmatrix} 0 & 3 & 5 & 6 \\ 7 & 0 & 2 & 3 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{pmatrix}.$$

$k = 4$: $d_{21} = \min(7, 3 + 2) = 5$; $d_{31} = \min(5, 1 + 2) = 3$; $d_{32} = \min(8, 1 + 5) = 6$.

$$D^{(4)} = \begin{pmatrix} 0 & 3 & 5 & 6 \\ 5 & 0 & 2 & 3 \\ 3 & 6 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{pmatrix} \quad (\text{final all-pairs shortest distances}).$$

Example: Three-vertex graph $D^{(0)} = \begin{pmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & \infty & 0 \end{pmatrix}$. $k = 1$: $d_{32} = 3 + 4 = 7$. $k = 2$: $d_{13} = 4 + 2 = 6$.

$k = 3$: $d_{21} = 2 + 3 = 5$. Final: $\begin{pmatrix} 0 & 4 & 6 \\ 5 & 0 & 2 \\ 3 & 7 & 0 \end{pmatrix}$.

To recover paths store a predecessor/intermediate matrix $\pi[i][j]$, updated whenever d_{ij} improves through k .

5.3 Travelling Salesperson Problem (Held–Karp)

Problem: given n cities and cost matrix c_{ij} , find the cheapest tour that visits every city exactly once and returns to the start. Brute force examines $(n - 1)!$ tours.

DP formulation. Fix city 1 as the start. Let $g(i, S)$ be the minimum cost of a path that starts at i , visits every city of the set S exactly once and ends at city 1:

$$g(i, \emptyset) = c_{i1}, \quad g(i, S) = \min_{j \in S} \{c_{ij} + g(j, S \setminus \{j\})\}, \quad \text{answer} = g(1, V \setminus \{1\}).$$

Complexity. There are $n \cdot 2^{n-1}$ values $g(i, S)$, each computed with $O(n)$ choices: $T(n) = O(n^2 2^n)$, space $O(n 2^n)$. Far better than $n!$ but still exponential (the problem is NP-hard).

Example: Four cities, cost matrix

$$C = \begin{pmatrix} 0 & 10 & 15 & 20 \\ 5 & 0 & 9 & 10 \\ 6 & 13 & 0 & 12 \\ 8 & 8 & 9 & 0 \end{pmatrix}.$$

Step 1 ($S = \emptyset$): $g(2, \emptyset) = c_{21} = 5$, $g(3, \emptyset) = 6$, $g(4, \emptyset) = 8$.

Step 2 ($|S| = 1$):

$$\begin{aligned} g(2, \{3\}) &= c_{23} + g(3, \emptyset) = 9 + 6 = 15, & g(2, \{4\}) &= c_{24} + g(4, \emptyset) = 10 + 8 = 18, \\ g(3, \{2\}) &= c_{32} + g(2, \emptyset) = 13 + 5 = 18, & g(3, \{4\}) &= c_{34} + g(4, \emptyset) = 12 + 8 = 20, \\ g(4, \{2\}) &= c_{42} + g(2, \emptyset) = 8 + 5 = 13, & g(4, \{3\}) &= c_{43} + g(3, \emptyset) = 9 + 6 = 15. \end{aligned}$$

Step 3 ($|S| = 2$):

$$\begin{aligned} g(2, \{3, 4\}) &= \min\{c_{23} + g(3, \{4\}), c_{24} + g(4, \{3\})\} = \min\{9 + 20, 10 + 15\} = 25 \ (j = 4), \\ g(3, \{2, 4\}) &= \min\{c_{32} + g(2, \{4\}), c_{34} + g(4, \{2\})\} = \min\{13 + 18, 12 + 13\} = 25 \ (j = 4), \\ g(4, \{2, 3\}) &= \min\{c_{42} + g(2, \{3\}), c_{43} + g(3, \{2\})\} = \min\{8 + 15, 9 + 18\} = 23 \ (j = 2). \end{aligned}$$

Step 4 (final):

$$g(1, \{2, 3, 4\}) = \min\{c_{12} + 25, c_{13} + 25, c_{14} + 23\} = \min\{35, 40, 43\} = 35 \ (j = 2).$$

Tour: from 1 go to 2; from $g(2, \{3, 4\})$ the best next is 4; from $g(4, \{3\})$ go to 3; then back to 1: $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$ with cost $10 + 10 + 9 + 6 = 35$. ✓

5.4 String Editing (Edit Distance)

String editing

Transform string $X = x_1..x_m$ into $Y = y_1..y_n$ with the minimum number (cost) of operations: **insert** a character, **delete** a character, **substitute** a character (each of cost 1; matching characters cost 0). The minimum is the **edit (Levenshtein) distance**.

Let $D[i][j]$ be the distance between the prefixes $X[1..i]$ and $Y[1..j]$.

$$D[i][0] = i, \quad D[0][j] = j, \quad D[i][j] = \min \begin{cases} D[i-1][j] + 1 & (\text{delete } x_i) \\ D[i][j-1] + 1 & (\text{insert } y_j) \\ D[i-1][j-1] + [x_i \neq y_j] & (\text{substitute / match}) \end{cases}$$

Complexity: $m \cdot n$ cells, $O(1)$ each: $T = O(mn)$; space $O(mn)$ (only two rows, $O(\min(m, n))$, if just the distance is needed).

Example: $X = \text{kitten}, Y = \text{sitting}$.

		s	i	t	t	i	n	g
	0	1	2	3	4	5	6	7
k	1	1	2	3	4	5	6	7
i	2	2	1	2	3	4	5	6
t	3	3	2	1	2	3	4	5
t	4	4	3	2	1	2	3	4
e	5	5	4	3	2	2	3	4
n	6	6	5	4	3	3	2	3

Sample cell: $D[6][7]$ (n vs g , mismatch) $= \min(D[5][7] + 1, D[6][6] + 1, D[5][6] + 1) = \min(5, 3, 4) = 3$.
Distance = 3. Traceback: kitten $\rightarrow$ sitten (substitute $k \rightarrow s$) $\rightarrow$ sittin (substitute $e \rightarrow i$) $\rightarrow$ sitting (insert g).

5.5 0/1 Knapsack Using Dynamic Programming

Problem: n items with weights w_i and profits p_i , capacity W ; choose a subset (each item whole or not at all) maximizing profit. Let $V[i][w]$ be the best profit using the first i items with capacity w :

$$V[0][w] = 0, V[i][0] = 0, \quad V[i][w] = \begin{cases} V[i-1][w] & w_i > w \\ \max(V[i-1][w], p_i + V[i-1][w - w_i]) & w_i \leq w \end{cases}$$

Traceback: start at $V[n][W]$; if $V[i][w] \neq V[i-1][w]$, item i is taken and $w \leftarrow w - w_i$; move to $i - 1$.

Complexity: $(n+1)(W+1)$ cells: $T = O(nW)$, $S = O(nW)$ ($O(W)$ if only the value is needed).
 $O(nW)$ is *pseudo-polynomial*: polynomial in the numeric value W but exponential in its bit-length.

```
def knapsack(w, p, W):
    n = len(w)
    V = [[0]*(W+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for c in range(W+1):
            V[i][c] = V[i-1][c]
            if w[i-1] <= c:
                V[i][c] = max(V[i][c], p[i-1] + V[i-1][c - w[i-1]])
    return V[n][W]
```

Example: $w = (2, 3, 4, 5)$, $p = (3, 5, 6, 10)$, $W = 8$ (the instance used for backtracking in Chapter 6).

$i \backslash w$	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1 ($w=2, p=3$)	0	0	3	3	3	3	3	3	3
2 ($w=3, p=5$)	0	0	3	5	5	8	8	8	8
3 ($w=4, p=6$)	0	0	3	5	6	8	9	11	11
4 ($w=5, p=10$)	0	0	3	5	6	10	10	13	15

Sample: $V[4][8] = \max(V[3][8], 10 + V[3][3]) = \max(11, 15) = 15$. Traceback: $V[4][8] = 15 \neq V[3][8] = 11 \Rightarrow$ take item 4, $w = 3$; $V[3][3] = 5 = V[2][3] \Rightarrow$ skip item 3; $V[2][3] = 5 \neq V[1][3] = 3 \Rightarrow$ take item 2, $w = 0$. Optimum: items $\{2, 4\}$, weight $3 + 5 = 8$, profit $5 + 10 = 15$.

Example: $w = (2, 3, 4, 5)$, $p = (3, 4, 5, 6)$, $W = 5$.

$i \backslash w$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

Maximum profit 7 with items $\{1, 2\}$ (weights $2 + 3 = 5$).

5.6 Matrix Chain Multiplication

Problem: multiply $A_1 A_2 \cdots A_n$ where A_i is $p_{i-1} \times p_i$. Multiplication is associative, so any parenthesization gives the same product but different cost: multiplying a $p \times q$ by a $q \times r$ matrix costs pqr scalar multiplications. The number of parenthesizations is the Catalan number $C_{n-1} = \frac{1}{n} \binom{2n-2}{n-1}$ (exponential).

Let $m[i, j]$ be the minimum cost of computing $A_i \cdots A_j$ and $s[i, j]$ the best split point k :

$$m[i, i] = 0, \quad m[i, j] = \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_{i-1} p_k p_j\} \quad (i < j).$$

Fill by increasing chain length $\ell = j - i + 1 = 2, 3, \dots, n$. Complexity: $O(n^2)$ cells, $O(n)$ choices each:

$$T = O(n^3), \quad S = O(n^2).$$

Example: $A_1: 10 \times 20$, $A_2: 20 \times 30$, $A_3: 30 \times 40$, $A_4: 40 \times 30$, so $p = (10, 20, 30, 40, 30)$.

Length 2: $m[1, 2] = 10 \cdot 20 \cdot 30 = 6000$; $m[2, 3] = 20 \cdot 30 \cdot 40 = 24000$; $m[3, 4] = 30 \cdot 40 \cdot 30 = 36000$.

Length 3:

- $m[1, 3]$: $k = 1$: $m[2, 3] + 10 \cdot 20 \cdot 40 = 24000 + 8000 = 32000$; $k = 2$: $m[1, 2] + 10 \cdot 30 \cdot 40 = 6000 + 12000 = 18000$. So $m[1, 3] = 18000$ ($k = 2$).
- $m[2, 4]$: $k = 2$: $m[3, 4] + 20 \cdot 30 \cdot 30 = 36000 + 18000 = 54000$; $k = 3$: $m[2, 3] + 20 \cdot 40 \cdot 30 = 24000 + 24000 = 48000$. So $m[2, 4] = 48000$ ($k = 3$).

Length 4:

- $k = 1$: $m[2, 4] + 10 \cdot 20 \cdot 30 = 48000 + 6000 = 54000$
- $k = 2$: $m[1, 2] + m[3, 4] + 10 \cdot 30 \cdot 30 = 6000 + 36000 + 9000 = 51000$
- $k = 3$: $m[1, 3] + 10 \cdot 40 \cdot 30 = 18000 + 12000 = 30000$

So $m[1, 4] = 30000$ with $k = 3$.

$m[i, j]$	$j = 1$	2	3	4	$s[i, j]$	$j = 2$	3	4
$i = 1$	0	6000	18000	30000	$i = 1$	1	2	3
2		0	24000	48000	2		2	3
3			0	36000	3			3
4				0				

Parenthesization: $s[1, 4] = 3 \Rightarrow (A_1 A_2 A_3) A_4$; $s[1, 3] = 2 \Rightarrow ((A_1 A_2) A_3) A_4$. Cost check: $A_1 A_2$: 6000 (result 10×30); $\times A_3$: $10 \cdot 30 \cdot 40 = 12000$ (10×40); $\times A_4$: $10 \cdot 40 \cdot 30 = 12000$; total 30000. ✓

5.7 Flow Shop Scheduling

Two-machine flow shop

n jobs must each be processed first on machine M_1 (time a_i) and then on machine M_2 (time b_i). A machine does one job at a time; a job cannot start on M_2 before it finishes on M_1 ; the same job order is used on both machines (a *permutation schedule*). Goal: minimize the **makespan** (finish time of the last job on M_2).

Dynamic-programming formulation. Let $g(S, t)$ be the minimum time to finish the set S of jobs when machine M_2 will still be busy for t time units after machine M_1 becomes free (time measured from that moment):

$$g(\emptyset, t) = t, \quad g(S, t) = \min_{i \in S} \left\{ a_i + g(S \setminus \{i\}, b_i + \max(t - a_i, 0)) \right\}, \quad \text{answer } g(\{1..n\}, 0).$$

(Choose job i to go next: M_1 is busy for a_i ; M_2 starts it at $\max(a_i, t)$ and is busy $b_i + \max(t - a_i, 0)$ after M_1 frees.) Time $O(n^2 2^n)$, but for two machines a greedy rule solves it optimally:

Johnson's rule

(1) Among all unscheduled jobs find the smallest processing time over both machines. (2) If it is on M_1 put that job at the *front* of the sequence; if it is on M_2 put it at the *end*. (3) Remove the job and repeat. Equivalently: jobs with $a_i \leq b_i$ first, in increasing a_i ; then jobs with $a_i > b_i$ in decreasing b_i . Time $O(n \log n)$.

Example: **Jobs** (a_i, b_i) : $J_1(3, 6)$, $J_2(5, 2)$, $J_3(1, 2)$, $J_4(6, 6)$, $J_5(7, 5)$.

- Minimum 1 (J_3 on M_1): front. Remaining J_1, J_2, J_4, J_5 .
- Minimum 2 (J_2 on M_2): end. Remaining J_1, J_4, J_5 .
- Minimum 3 (J_1 on M_1): front (after J_3). Remaining J_4, J_5 .
- Minimum 5 (J_5 on M_2): end (before J_2). Remaining J_4 : middle.

Sequence: J_3, J_1, J_4, J_5, J_2 .

Job	J_3	J_1	J_4	J_5	J_2
M_1 (start–end)	0–1	1–4	4–10	10–17	17–22
M_2 (start–end)	1–3	4–10	10–16	17–22	22–24

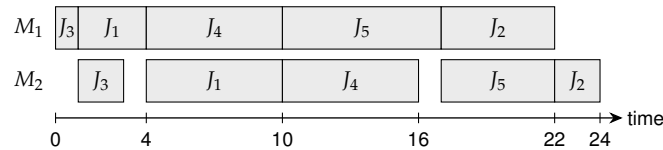


Figure 7: Gantt chart of Johnson's schedule; makespan = 24.

Makespan = **24** (equal to $\sum a_i + b_{\text{last}} = 22 + 2$, which is a lower bound, so the schedule is optimal). For $m \geq 3$ machines the problem is NP-hard.

Chapter 5 recap. Floyd–Warshall $\Theta(n^3)$ time, $\Theta(n^2)$ space; Held–Karp TSP $O(n^2 2^n)$; edit distance $O(mn)$; 0/1 knapsack $O(nW)$ (pseudo-polynomial); matrix chain $O(n^3)$; two-machine flow shop optimal by Johnson's rule in $O(n \log n)$.

Chapter 6 Backtracking Techniques

6.1 Basic Concepts

Backtracking

A systematic, refined brute-force technique that builds a solution *incrementally*, one component at a time, and **abandons** (backtracks from) a partial solution as soon as it is certain that it cannot be completed to a valid one. It is a depth-first traversal of the **state-space tree**: the root is the empty partial solution, internal nodes are partial solutions, *dead ends* violate a constraint, and *goal nodes* are complete valid solutions.

Steps: (1) *Choose* an option at the current state; (2) *Explore* by recursing; (3) *Check validity* with a **bounding (feasibility) function** and prune if violated; (4) *Backtrack*: undo the choice and try the next; (5) *Success*: record a complete valid solution.

```
def backtrack(partial):
    if is_solution(partial): report(partial); return
    for choice in candidates(partial):
        if feasible(partial, choice): # bounding function (pruning)
            partial.append(choice)    # choose
            backtrack(partial)        # explore
            partial.pop()             # undo = backtrack
```

Applications: Sudoku, *N*-Queens, graph colouring, Hamiltonian cycle, sum of subsets, 0/1 knapsack, maze/knight's tour. **Advantages:** prunes early (faster than pure brute force); guaranteed to find a solution if one exists; simple recursive code. **Disadvantages:** worst case exponential; may explore many dead ends; does not by itself give an optimum unless all solutions are examined.

Recursion	Backtracking
General technique: a function calls itself on smaller subproblems	Algorithmic technique built on recursion for constraint problems
Need not explore alternatives (factorial, Hanoi, merge sort)	Always explores alternatives and undoes decisions (<i>N</i> -Queens, Sudoku)

Every backtracking algorithm is recursive, but not every recursive algorithm is backtracking.

6.2 The *N*-Queens Problem

N-Queens

Place *N* queens on an $N \times N$ board so that no two attack each other: no two share a row, column or diagonal. Queens at (r_i, c_i) and (r_j, c_j) conflict iff $c_i = c_j$ or $|r_i - r_j| = |c_i - c_j|$ (same row is avoided by placing exactly one queen per row).

Algorithm. Place queen *k* in row *k*; try columns 1..*N*; accept the first column with no conflict with queens 1..*k* − 1 and go to row *k* + 1; if no column works, backtrack to row *k* − 1 and move that queen to its next column; a solution is found when all *N* rows are filled.

```
def place(x, k, c):
    # x[1..k-1] already placed; try column c in row k
    return all(x[j] != c and abs(x[j] - c) != abs(j - k) for j in range(1, k))

def nqueens(x, k, n):
    for c in range(1, n + 1):
        if place(x, k, c):
            x[k] = c
```

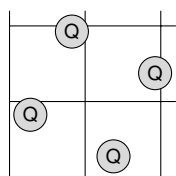
```

if k == n: print(x[1:])
else:     nqueens(x, k + 1, n)

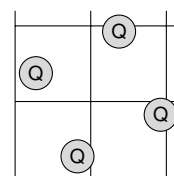
```

Example: 4-Queens trace (board state = column of the queen in each row).

Step	Action	State
1	Q_1 at (1,1)	(1, -, -, -)
2	Q_2 : col 1 (same column), col 2 (diagonal) fail; col 3 ok	(1, 3, -, -)
3	Q_3 : cols 1, 2, 3, 4 all conflict → backtrack to Q_2	dead end
4	move Q_2 to col 4	(1, 4, -, -)
5	Q_3 at col 2 ok	(1, 4, 2, -)
6	Q_4 : all columns conflict → backtrack ; Q_3 has no more columns, back to Q_2 (none), back to Q_1	dead end
7	move Q_1 to col 2; Q_2 col 4; Q_3 col 1; Q_4 col 3	(2, 4, 1, 3) ✓
8	continue: Q_1 at col 3; Q_2 col 1, Q_3 col 4, Q_4 col 2	(3, 1, 4, 2) ✓
9	Q_1 at col 4: no completion; all branches exhausted	done



Solution 1: (2, 4, 1, 3)



Solution 2: (3, 1, 4, 2)

Figure 8: The two solutions of 4-Queens (row 1 at the top).

The state-space tree of 4-Queens has root with 4 children; the branches $Q_1=1$ and $Q_1=4$ die, while $Q_1=2$ and $Q_1=3$ each lead to exactly one solution. Known counts: $N = 1, 4, 5, 6, 7, 8$ have 1, 2, 10, 4, 40, 92 solutions ($N = 2, 3$ have none); 8-Queens has 92 solutions (12 distinct up to symmetry).

Time: row 1 has n choices, row 2 at most $n - 1, \dots$: $T(n) \approx n!$, $T(n) = O(n!)$ (pruning cuts the real count far below this, versus n^n or $\binom{n^2}{n}$ for brute force). **Space:** recursion depth n plus the array $x[1..n]$ (or three Boolean arrays for columns and the two diagonals), $S(n) = O(n)$; storing the whole board would be $O(n^2)$.

6.3 Sum of Subsets

Sum of subsets

Given n positive integers $W = \{w_1 < w_2 < \dots < w_n\}$ (sorted ascending) and a target M , find all subsets whose sum equals M ; i.e. binary vectors x with $\sum w_i x_i = M$, $x_i \in \{0, 1\}$.

State-space tree: level i decides x_i : left branch include w_i ($x_i = 1$), right branch exclude ($x_i = 0$). Let sum be the total chosen so far and rem the sum of the remaining items. **Bounding conditions:** prune if (1) $sum > M$ (over-shoot), or (2) $sum + rem < M$ (under-shoot: even taking everything left cannot reach M). If $sum = M$ a solution is printed.

```

def subset_sum(i, s, rem, W, M, x):
    # items 0..n-1, W sorted
    if s + rem < M or s > M: return # feasibility / pruning
    if s == M: print([W[j] for j in range(i) if x[j]]); return
    x[i] = 1; subset_sum(i+1, s + W[i], rem - W[i], W, M, x) # include
    x[i] = 0; subset_sum(i+1, s, rem - W[i], W, M, x) # exclude

```

Time: the unpruned tree has $1 + 2 + 2^2 + \dots + 2^n = 2^{n+1} - 1$ nodes: $T(n) = O(2^n)$ in the worst case.

Space: vector x and stack depth n : $S(n) = O(n)$ (excluding stored output).

Example: $n = 7$, $W = (5, 7, 10, 12, 15, 18, 20)$, $M = 35$. Total = 87. The search tree (sum at each node; prune when > 35 or when $sum + rem < 35$) finds four solutions: $(5, 10, 20)$, $(5, 12, 18)$, $(7, 10, 18)$, $(15, 20)$. Partial trace of the left-most branch: $\{5\} (5) \rightarrow \{5, 7\} (12) \rightarrow \{5, 7, 10\} (22) \rightarrow \{5, 7, 10, 12\} (34)$; adding 15: $49 > 35$ prune; skipping 15, adding 18: $52 > 35$ prune; adding 20: $54 > 35$ prune; no more items, backtrack and remove 12. Then $\{5, 7, 10\} (22) + 15 = 37 > 35$, $+18 = 40 > 35$, $+20 = 42 > 35$: prune; backtrack to $\{5, 7\}$; exclude 10, include 12: $\{5, 7, 12\} (24) + 15 = 39 > 35$ prune, $+18 = 42$, $+20 = 44$ prune; ... later $\{5, 10\} (15) \rightarrow \{5, 10, 20\}$ gives $35 = M$: **solution**. The remaining three are found when the search continues through all branches.

Example: (Question 6.1) $n = 4$, $W = (11, 13, 24, 7)$, $M = 31$. Sort ascending: $(7, 11, 13, 24)$, total 55.

Partial subset	sum / rem	Decision
$\{7\}$	7 / 48	continue
$\{7, 11\}$	18 / 37	continue
$\{7, 11, 13\}$	31 / 24	$sum = M$: solution
$\{7, 11, 24\}$	42	> 31 : prune
$\{7, 11\}$ (skip 13, 24)	18 / 0	$18 < 31$: prune
$\{7, 13\}$	20 / 24	include 24: $44 > 31$ prune; skip 24: $20 < 31$ prune
$\{7, 24\}$	31	solution
$\{11 \dots\}$ (exclude 7)	11 / 37	$\{11, 13\} = 24$, $+24 = 48 > 31$; $\{11, 24\} = 35 > 31$; both prune
$\{13, 24\}$	37	prune; remaining < 31 : prune

Solutions: $\{7, 11, 13\}$ and $\{7, 24\}$; in the original order $(11, 13, 24, 7)$: $x = (1, 1, 0, 1)$ and $x = (0, 0, 1, 1)$.

6.4 Graph Colouring

Graph colouring (vertex colouring)

Colour the vertices of a graph so that no two adjacent vertices get the same colour. **Decision version:** can G be coloured with m colours? **Optimization:** find the minimum number of colours, the **chromatic number** $\chi(G)$.

Algorithm. Colour vertex 1 with colour 1. For each next vertex try colours $1..m$ and accept the first one not used by an already-coloured neighbour; if none works, backtrack to the previous vertex and increment its colour.

```
def is_safe(adj, color, i, c):
    return all(not (adj[i][j] and color[j] == c) for j in range(i))

def graph_coloring(adj, m, i, color): # prints every valid colouring
    n = len(adj)
    if i == n: print(color); return
    for c in range(1, m + 1):
        if is_safe(adj, color, i, c):
            color[i] = c
            graph_coloring(adj, m, i + 1, color)
            color[i] = 0 # backtrack
```

Time: at most m^V complete assignments; the tree has $1 + m + \dots + m^V = \frac{m^{V+1}-1}{m-1}$ nodes, and each `is_safe` costs $O(V)$: $T = O(V \cdot m^V)$. **Space:** `COLOR[]` $O(V)$ plus recursion depth V : $S = O(V)$ (graph storage extra: $O(V^2)$ matrix or $O(V + E)$ list).

Example: Graph with vertices 1..4 and edges 12, 23, 34, 41, 13 (a square with the diagonal 13), $m = 3$. $1 \rightarrow$ colour 1. Vertex 2 (adjacent to 1): colour 1 clashes, colour 2 ok. Vertex 3 (adjacent to 1, 2, 4): colours 1, 2 clash, colour 3 ok. Vertex 4 (adjacent to 3 and 1): colour 1 (clash with vertex 1), colour 2 ok

→ (1, 2, 3, 2) is a valid colouring.

Counting all: vertex 1 has 3 choices, vertex 2 has 2, vertex 3 has 1 (triangle 1, 2, 3), vertex 4 has 1 (must differ from the two different colours of vertices 1 and 3): $3 \cdot 2 \cdot 1 \cdot 1 = 6$ colourings, e.g. (1, 2, 3, 2), (1, 3, 2, 3), (2, 1, 3, 1), ... With $m = 2$: vertex 3 finds no colour (triangle), so the search backtracks everywhere and reports failure: $\chi(G) = 3$.

6.5 Hamiltonian Cycle

Hamiltonian cycle

A closed path in an undirected graph $G = (V, E)$ that visits every vertex *exactly once* and returns to the start (it need not use every edge). The problem “does G have a Hamiltonian cycle?” is NP-complete.

A candidate sequence $v_0, \dots, v_{n-1}$ is valid iff: **C1 adjacency** – consecutive vertices are joined by an edge; **C2 closure** – v_{n-1} is adjacent to v_0 ; **C3 uniqueness** – no repeated vertex.

Algorithm. Fix vertex 1 as $V[0]$. To fill position i try every vertex j ; accept it if FEASIBLE: j is not already in the path and j is adjacent to $V[i-1]$. When $i = n-1$ also require adjacency of $V[n-1]$ to $V[0]$, then print the cycle; otherwise recurse to position $i+1$; if no vertex fits, backtrack.

```
def hamiltonian(adj, path, i):
    n = len(adj)
    for j in range(1, n):
        if j not in path[:i] and adj[path[i-1]][j]: # feasible
            path[i] = j
            if i == n - 1:
                if adj[path[n-1]][path[0]]: print(path + [path[0]])
            else:
                hamiltonian(adj, path, i + 1)
# call with path = [0] + [None]*(n-1), i = 1
```

Time: level 1 has $\leq n-1$ choices, level 2 $\leq n-2, \dots: (n-1)!$ paths, $O(n!)$. **Space:** path array and recursion, $O(n)$.

Example: Eight-vertex graph with edges 12, 13, 23, 25, 16, 34, 37, 48, 56, 67, 78. By backtracking from vertex 1: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 8 \rightarrow 7 \rightarrow 6 \rightarrow 5$, then no edge $5 \rightarrow 1$: dead end; backtrack (6, 7, 8, 4) to 3; try $3 \rightarrow 7 \rightarrow 8 \rightarrow 4$: dead end; $3 \rightarrow 7 \rightarrow 6 \rightarrow 5$: dead end; backtrack to 2; try $2 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 4 \rightarrow 3 \rightarrow 1$: **cycle found** 1, 2, 5, 6, 7, 8, 4, 3, 1. Traversing in the other direction ($1 \rightarrow 3 \rightarrow 4 \rightarrow 8 \rightarrow 7 \rightarrow 6 \rightarrow 5 \rightarrow 2 \rightarrow 1$) gives the same cycle reversed. Check by degrees: vertices 5, 4, 8 have degree 2, forcing the paths 2–5–6 and 3–4–8–7; closing the cycle then forces 7–6 and 1 between 2 and 3, so this undirected cycle is unique.

Example: Six-vertex graph $a..f$ with edges $ab, ac, ad, bc, bf, cd, ce, de, ef$. Vertex f has degree 2, so every cycle contains $b-f-e$. Backtracking from a finds, e.g., a, b, f, e, c, d, a ; a, b, f, e, d, c, a and a, c, b, f, e, d, a (and their reversals) – **three** undirected Hamiltonian cycles. Branches such as $a, b, c, \dots$ die when f cannot be reached properly.

6.6 0/1 Knapsack by Backtracking

Maximize $\sum p_i x_i$ subject to $\sum w_i x_i \leq W$, $x_i \in \{0, 1\}$. State-space tree: at level i include item i (only if $cw + w_i \leq W$) or exclude it. Keep the best profit found so far.

Algorithm: Knapsack($i, cw, cp, currentSet$)

If $cw \leq W$ and $cp > maxProfit$: $maxProfit \leftarrow cp$, $bestSet \leftarrow currentSet$. If $i = n$: return. If $cw + w[i] \leq W$: Knapsack($i+1, cw + w[i], cp + p[i], currentSet \cup \{i\}$) (include). Knapsack($i+1, cw, cp, currentSet$) (exclude). Start with Knapsack(1, 0, 0, $\emptyset$), $maxProfit = 0$.

Time: binary tree with $2^{n+1} - 1$ nodes: $O(2^n)$. **Space:** depth n plus *currentSet* and *bestSet*: $O(n)$. (Adding an upper bound such as the fractional-knapsack value of the remaining items lets us prune branches that cannot beat *maxProfit*: *branch and bound*.)

Example: $w = (2, 3, 4, 5)$, $p = (3, 5, 6, 10)$, $W = 8$. Nodes written ($cw \mid cp$):

Branch	Result
{1, 2, 3}	(9 14) infeasible ($9 > 8$)
{1, 2, 4}	(10 18) infeasible
{1, 2}	(5 8) feasible, best so far 8
{1, 3}	(6 9) feasible, best 9; {1, 3, 4} (11) infeasible
{1, 4}	(7 13) feasible, best 13
{2, 3}	(7 11); {2, 3, 4} infeasible; {2, 4} (8 15) feasible, best 15
{3, 4}	(9 16) infeasible ($4 + 5 = 9 > 8$)

Maximum profit = **15** with items {2, 4} (weight $3 + 5 = 8$), the same optimum as the dynamic-programming table of Chapter 5. (The lecture figure labels the chosen items I_3, I_4 ; with the data above the feasible optimal pair is $\{I_2, I_4\}$, since $w_3 + w_4 = 9 > 8$.)

Example: (Question 6.2) $w = (2, 3, 5, 7)$, $p = (3, 4, 8, 10)$, $W = 10$. Feasible full sets: {1, 2, 3} weight 10, profit 15; {2, 4} weight 10, profit 14; {1, 4} weight 9, profit 13; {1, 3} (7 | 11); {3} plus anything else of weight ≤ 5 is only item 1 or 2. Maximum profit **15**, items {1, 2, 3}.

Chapter 6 recap. Backtracking = DFS of the state-space tree with a bounding function. *N*-Queens $O(n!)$; sum of subsets $O(2^n)$ (prune $sum > M$ and $sum + rem < M$); graph colouring $O(V m^V)$; Hamiltonian cycle $O(n!)$; 0/1 knapsack $O(2^n)$. Space is $O(n)$ or $O(V)$ for all of them (recursion depth).

Chapter 7 NP-Hard and NP-Complete Problems

7.1 Basic Concepts

Tractable vs intractable. A problem is **tractable** if some algorithm solves it in polynomial time $O(n^k)$; the problems of Chapters 1–6 (sorting, MST, shortest paths, GCD) are of this kind. Problems whose best known algorithms are exponential (2^n , $n!$) are **intractable** in practice.

Deterministic and nondeterministic algorithms

A **deterministic** algorithm has a single outcome for each step. A **nondeterministic** algorithm may use three extra operations: `choice(S)` (arbitrarily picks an element of S in $O(1)$), `failure()` (unsuccessful termination) and `success()` (successful termination). It always makes the choices that lead to success if any exist, so it can be thought of as “guess a solution, then verify it”. A nondeterministic algorithm is **polynomial** if the shortest successful computation takes polynomial time.

Example: Nondeterministic search for x in $A[1..n]$: `j = choice(1..n); if A[j]==x then success()` else `failure()` – time $O(1)$ versus $\Theta(n)$ deterministically. *Example: Nondeterministic sort*: guess a permutation, verify in $O(n)$ that it is sorted.

Classes P and NP (decision problems)

P = decision problems solvable by a deterministic algorithm in polynomial time. **NP** = decision problems solvable by a nondeterministic algorithm in polynomial time; equivalently, problems whose “yes” answers can be *verified* in polynomial time given a certificate (proof). Clearly $P \subseteq NP$. Whether $P = NP$ is the most famous open problem in computer science.

Decision vs optimization. NP is defined for yes/no problems. An optimization problem is converted to a decision problem by adding a bound: “is there a tour of cost $\leq k$?”, “is there a clique of size $\geq k$?”. If the decision version is hard, so is the optimization version.

In P	NP-complete (counterpart)	Difference
Shortest simple path	Longest simple path	min vs max
Euler cycle (every edge once)	Hamiltonian cycle (every vertex once)	edge vs vertex
2-SAT	3-SAT	clause size 2 vs 3
2-colouring, bipartiteness	3-colouring	colours 2 vs 3
Minimum spanning tree	Steiner tree, degree-constrained ST	extra constraints
Linear programming (real)	0/1 integer programming	real vs integer

Reducibility, NP-hard and NP-complete

Polynomial-time reduction

Problem L_1 **reduces** to L_2 ($L_1 \leq_p L_2$) if there is a polynomial-time algorithm that transforms any instance I_1 of L_1 into an instance I_2 of L_2 such that the answer to I_1 is “yes” iff the answer to I_2 is “yes”. Thus if L_2 is solvable in polynomial time, so is L_1 ; conversely if L_1 is hard then L_2 is at least as hard. $\leq_p$ is transitive.

NP-hard and NP-complete

L is **NP-hard** if every problem in NP reduces to it ($L' \leq_p L$ for all $L' \in NP$); L need not be in NP (e.g. the halting problem; optimization versions). L is **NP-complete** if (1) $L \in NP$ and (2) L is NP-hard.

NP-complete problems are the hardest problems in NP: a polynomial algorithm for any one gives $P = NP$.

To prove L NP-complete: (i) show $L \in NP$ (give a polynomial verifier); (ii) pick a known NP-complete problem L' and show $L' \leq_p L$.

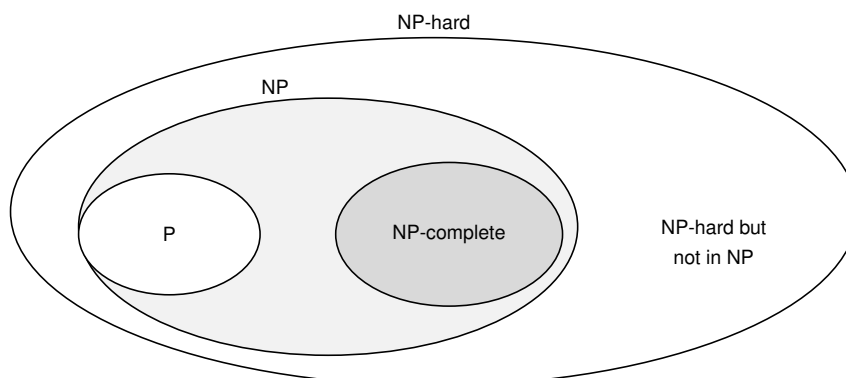


Figure 9: Classes if $P \neq NP$. $NP\text{-complete} = NP \cap NP\text{-hard}$. (If $P = NP$ then $P = NP = NP\text{-complete}$.)

7.2 Cook's Theorem

Satisfiability (SAT) and Cook's theorem

SAT: given a Boolean formula in conjunctive normal form (an AND of clauses, each an OR of literals), is there an assignment of truth values making it true? **Cook's theorem (Cook–Levin, 1971):** SAT is NP-complete; i.e. $SAT \in P$ iff $P = NP$.

Why SAT is in NP: guess an assignment and evaluate the formula in linear time.

Proof idea for NP-hardness. Let $L \in NP$ be decided by a nondeterministic Turing machine M in time $p(n)$ on input x . Construct in polynomial time a CNF formula $F(x)$ whose variables describe, for every time step $t \leq p(n)$ and tape position, the tape symbol, the head position and the machine state. Clauses force: (a) the initial configuration encodes x ; (b) every consecutive pair of configurations follows a legal transition of M ; (c) the final configuration is accepting; (d) exactly one symbol per cell and one state per time. The formula has $O(p(n)^2)$ variables and clauses, and $F(x)$ is satisfiable iff M accepts x . Hence every $L \in NP$ reduces to SAT.

Consequence: once one NP-complete problem is known, others are proved by reduction (chain below).

$$SAT \leq_p 3\text{-SAT} \leq_p \text{Clique} \leq_p \text{Vertex Cover} \leq_p \text{Hamiltonian Cycle} \leq_p \text{TSP}$$

Example: 3-SAT $\leq_p$ Clique. Formula $F = C_1 \wedge \dots \wedge C_k$ with 3 literals per clause. Build a graph with one vertex per literal occurrence; join two vertices iff they belong to *different* clauses and are not contradictory (x vs $\bar{x}$). F is satisfiable iff the graph has a clique of size k (one true literal from each clause, mutually consistent). The graph has $3k$ vertices: polynomial.

7.3 NP-Hard Graph Problems

Problem	Statement (decision version)
Clique	Does G contain a complete subgraph on k vertices?
Independent set	Is there a set of k pairwise non-adjacent vertices?
Vertex (node) cover	Is there a set of $\leq k$ vertices touching every edge?
Hamiltonian cycle	Is there a simple cycle through every vertex?
Travelling salesperson	Is there a tour visiting all cities of total cost $\leq k$?
Graph colouring	Can the vertices be coloured with $\leq k$ colours, adjacent ones different? (NP-complete for $k \geq 3$)
Chromatic number	Minimum number of colours (optimization, NP-hard)

Reductions among them.

- **Clique $\equiv$ Independent set:** S is a clique in G iff S is an independent set in the complement graph $\bar{G}$.
- **Independent set $\equiv$ Vertex cover:** S is independent iff $V \setminus S$ is a vertex cover; so G has an independent set of size k iff it has a vertex cover of size $|V| - k$.
- **Hamiltonian cycle $\leq_p$ TSP:** given G with n vertices build the complete graph with cost 1 on edges of G and 2 on non-edges; G has a Hamiltonian cycle iff the complete graph has a tour of cost $\leq n$.

Verification in polynomial time (so they are in NP): a proposed clique, cover or colouring is checked in $O(n^2)$; a tour in $O(n)$.

7.4 NP-Hard Scheduling Problems

- **Partition / Subset sum:** can a set of integers be split into two parts of equal sum? (NP-complete; pseudo-polynomial DP exists, Chapter 5.)
- **Multiprocessor scheduling:** assign n jobs with times t_i to $m \geq 2$ identical processors to minimize the finish time (makespan); for $m = 2$ it is Partition, hence NP-hard.
- **Flow shop with $m \geq 3$ machines and job shop scheduling** (minimize makespan): NP-hard (two-machine flow shop is polynomial by Johnson's rule).
- **Scheduling with deadlines and non-unit times** ("sequencing within intervals", minimizing weighted tardiness): NP-hard; the unit-time version of Chapter 4 is polynomial.
- **Bin packing and 3-Partition:** NP-hard (strongly), no pseudo-polynomial algorithm unless $P = NP$.

Good practical heuristic (approximation): Graham's *list scheduling* puts each job on the currently least-loaded processor: makespan $\leq (2 - \frac{1}{m}) OPT$; processing jobs in decreasing order of length (*LPT*) improves this to $\leq (\frac{4}{3} - \frac{1}{3m}) OPT$.

7.5 NP-Hard Code Generation Problems

Generating *optimal* machine code for an expression is a combinatorial problem: choose an evaluation order and decide which values stay in registers so that the number of instructions (loads/stores) is minimum.

- For an expression *tree* with n registers, the minimum is found in linear time by the **Sethi–Ullman** labelling: leaf label 1 (left) or 0 (right operand leaf); for an interior node with children labels l, r : label = $\max(l, r)$ if $l \neq r$, else $l + 1$. The label is the number of registers needed; evaluate the child with the larger label first.
- For an expression *DAG* with common subexpressions, generating optimal code with k registers is **NP-hard** (Aho–Johnson–Ullman, also for a single register), and so is optimal register allocation (equivalent to graph colouring), and optimal scheduling of instructions with a limited number of registers.

Example: $(a + b) * (c - d)$: leaves a (1), b (0) $\Rightarrow$ + node: labels differ, $\max = 1$; similarly $-$ node label 1; root: $l = r = 1 \Rightarrow$ label 2. Two registers suffice: $R1 = a + b$; $R2 = c - d$; $R1 = R1 * R2$.

7.6 Simplified NP-Hard Problems

Restricting an NP-hard problem can either keep it NP-hard (useful for proofs: simpler instances to reduce from) or make it easy.

- **3-SAT** (each clause has exactly 3 literals) is NP-complete, but **2-SAT** is in P (implication graph, linear time).
- **Vertex cover** on bipartite graphs is polynomial (König's theorem), but on general graphs NP-hard; **clique** on planar graphs is polynomial (no K_5), **3-colouring** of planar graphs is NP-complete while 4-colouring always exists.
- **Hamiltonian cycle** stays NP-complete even for planar cubic graphs and for undirected graphs of maximum degree 3.
- **Subset sum / Partition / Knapsack** are weakly NP-hard: polynomial when the numbers are small (pseudo-polynomial $O(nW)$).
- **TSP** stays NP-hard for metric (triangle-inequality) and Euclidean instances, but then constant-factor approximations exist.

7.7 Approximation Algorithms

Approximation ratio and ε -approximation

For a minimization problem an algorithm returning cost C (optimum C^*) has **approximation ratio** ρ if $C \leq \rho C^*$ for every instance (for maximization $C \geq C^* / \rho$). An **absolute** ε -approximation satisfies $|C - C^*| \leq \varepsilon$; a **relative** one satisfies $|C - C^*| / C^* \leq \varepsilon$. Instead of the exact optimum we accept a provably near-optimal answer in polynomial time.

- **PTAS** (polynomial-time approximation scheme): for every fixed $\varepsilon > 0$ an algorithm giving ratio $1 + \varepsilon$ in time polynomial in n (may be $n^{1/\varepsilon}$).
- **FPTAS** (fully PTAS): running time polynomial in both n and $1/\varepsilon$.
- **Probabilistically good algorithms**: randomized algorithms that run in polynomial time and give a correct or near-optimal answer with high probability, or whose *expected* performance is good on random inputs (e.g. Monte Carlo algorithms such as Miller–Rabin, random-pivot quicksort, greedy colouring that is optimal for almost all random graphs). They trade certainty for speed: the failure probability can be driven down by repetition.

Vertex cover problem: a 2-approximation

APPROX-VERTEX-COVER(G)

$C \leftarrow \emptyset$; $E' \leftarrow E$. While $E' \neq \emptyset$: pick any edge $(u, v) \in E'$; add *both* u and v to C ; delete from E' every edge incident to u or v . Return C .

Time $O(|V| + |E|)$. **Ratio** ≤ 2 : C is a cover (every edge is removed only when one endpoint is in C). The picked edges A form a *matching* (no two share a vertex), so any cover, in particular an optimal one C^* , must contain at least one endpoint of each, hence $|C^*| \geq |A|$. But $|C| = 2|A| \leq 2|C^*|$.

Example: Edges ab, bc, cd, de, ef, dg . Pick (b, c) : remove ab, bc, cd . Pick (e, f) : remove de, ef . Pick (d, g) : remove dg . $C = \{b, c, e, f, d, g\}$, size 6. Optimum $\{b, d, e\}$ has size 3: ratio exactly 2.

Subset sum problem: an FPTAS

Given positive integers $S = \{x_1 \dots x_n\}$ and target t , find the subset with the largest sum $\leq t$. Exact: maintain the list L_i of all achievable sums (size up to 2^i). **Trimming** with parameter $\delta = \frac{\varepsilon}{2n}$: scan the sorted list and drop y if the last kept value z satisfies $y \leq z(1 + \delta)$ (it is already represented within

factor $1 + \delta$). Each step: $L_i = \text{TRIM}(\text{MERGE}(L_{i-1}, L_{i-1} + x_i), \delta)$ with elements $> t$ removed. The final answer is the largest value in L_n .

Guarantee: the result z^* satisfies $z^* \geq \frac{y^*}{1+\varepsilon}$ (y^* optimal); list lengths are $O(\frac{n \ln t}{\varepsilon})$, so the running time is $O(\frac{n^2 \ln t}{\varepsilon})$, polynomial in n and $1/\varepsilon$: an **FPTAS**.

Example: $S = \{104, 102, 201, 101\}$, $t = 308$, $\varepsilon = 0.4$, so $\delta = \frac{0.4}{8} = 0.05$.

i	x_i	L_i after merge, trim ($y \leq z \cdot 1.05$ dropped) and removing > 308
0	–	$\langle 0 \rangle$
1	104	$\langle 0, 104 \rangle$
2	102	merged $\langle 0, 102, 104, 206 \rangle$; $104 \leq 102 \cdot 1.05 = 107.1$ dropped $\rightarrow \langle 0, 102, 206 \rangle$
3	201	merged $\langle 0, 102, 201, 206, 303, 407 \rangle$; $206 \leq 201 \cdot 1.05 = 211.05$ dropped; $407 > 308$ removed $\rightarrow \langle 0, 102, 201, 303 \rangle$
4	101	merged $\langle 0, 101, 102, 201, 203, 302, 303, 404 \rangle$; $102 \leq 106.05$, $203 \leq 211.05$, $303 \leq 317.1$ dropped; 404 removed $\rightarrow \langle 0, 101, 201, 302 \rangle$

Result 302. The true optimum is $104 + 102 + 101 = 307$; $302 \geq \frac{307}{1.4} = 219.3$ (in fact within 1.7%). ✓

Other classical approximations

- **Metric TSP, 2-approximation:** build an MST (cost $\leq OPT$, since removing one edge of the optimal tour gives a spanning tree), walk it in preorder, shortcut repeated vertices (triangle inequality): tour cost $\leq 2 \cdot MST \leq 2OPT$. Christofides improves to 1.5. For general (non-metric) TSP no constant ratio is possible unless $P = NP$.
- **Greedy set cover:** ratio $\ln n + 1$. **Max-SAT** random assignment: expected $\geq \frac{7}{8}$ of clauses satisfied for 3-SAT.
- **Bin packing first-fit:** $\leq 1.7OPT$; **knapsack:** FPTAS by scaling profits.

Chapter 7 recap. $P \subseteq NP$; NP-complete = $NP \cap NP\text{-hard}$. Prove NP-completeness by (i) membership in NP, (ii) a polynomial reduction from a known NP-complete problem. Cook: SAT is NP-complete; chain $\text{SAT} \rightarrow 3\text{-SAT} \rightarrow \text{Clique} \rightarrow \text{Vertex cover} \rightarrow \text{Hamiltonian} \rightarrow \text{TSP}$. Cope with hardness by approximation: vertex cover 2-approx, metric TSP 2-approx, subset sum FPTAS ($\delta = \varepsilon/2n$), PTAS/FPTAS definitions.

Summary and Knowledge Check

Complexity summary

Algorithm	Time	Space	Paradigm
Euclid GCD	$O(\log \min(a, b))$	$O(1)$	iterative
Fibonacci (iterative)	$O(n)$	$O(1)$	iterative
Linear / binary search	$O(n) / O(\log n)$	$O(1)$	–/D&C
Bubble, selection, insertion	$O(n^2)$ (insertion/bubble best $O(n)$)	$O(1)$	iterative
Extended Euclid	$O(\log \min(a, b))$	$O(\log \min(a, b))$	recursive
Miller–Rabin	$O(k \log n)$ mults	$O(1)$	randomized
Merge sort	$\Theta(n \log n)$	$O(n)$	D&C
Heap sort	$\Theta(n \log n)$	$O(1)$	D&C / heap
Quick sort	$O(n \log n)$ avg, $O(n^2)$ worst	$O(\log n)$	D&C
Select (rand. / median of medians)	$O(n)$ expected / worst	$O(\log n)$	D&C
Fractional knapsack	$O(n \log n)$	$O(1)$	greedy
Job sequencing	$O(n^2)$	$O(n)$	greedy
Kruskal / Prim	$O(E \log V) / O(V^2)$	$O(V + E)$	greedy
Dijkstra	$O(V^2)$ or $O((V + E) \log V)$	$O(V)$	greedy
Floyd–Warshall	$\Theta(n^3)$	$\Theta(n^2)$	DP
TSP (Held–Karp)	$O(n^2 2^n)$	$O(n 2^n)$	DP
Edit distance	$O(mn)$	$O(mn)$	DP
0/1 knapsack (DP)	$O(nW)$	$O(nW)$	DP
Matrix chain	$O(n^3)$	$O(n^2)$	DP
N -Queens, Hamiltonian	$O(n!)$	$O(n)$	backtracking
Sum of subsets, 0/1 knapsack (BT)	$O(2^n)$	$O(n)$	backtracking
Graph colouring	$O(V m^V)$	$O(V)$	backtracking

30 multiple-choice questions

- $f(n) = 3n + 2$ is: (A) $O(1)$ (B) $\Theta(n)$ (C) $\Theta(n^2)$ (D) $\Omega(n^2)$
- The solution of $T(n) = 2T(n/2) + n$ is: (A) $\Theta(n)$ (B) $\Theta(n^2)$ (C) $\Theta(n \log n)$ (D) $\Theta(\log n)$
- $T(n) = 9T(n/3) + n$ gives: (A) $\Theta(n)$ (B) $\Theta(n^2)$ (C) $\Theta(n^2 \log n)$ (D) $\Theta(n^3)$
- Aggregate analysis of n stack operations including MULTIPOP gives amortized cost per operation: (A) $O(n)$ (B) $O(\log n)$ (C) $O(1)$ (D) $O(n^2)$
- The number of iterations of Euclid's algorithm is worst when the inputs are: (A) powers of 2 (B) consecutive Fibonacci numbers (C) primes (D) equal
- Using extended Euclid, $198x + 128y = 2$ has $x, y =$: (A) 11, –17 (B) –17, 11 (C) 17, –11 (D) 1, –1
- $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$ gives $x \pmod{15} =$: (A) 5 (B) 8 (C) 13 (D) 11
- $3^{100} \pmod{7} =$: (A) 1 (B) 2 (C) 3 (D) 4
- $15x \equiv 35 \pmod{50}$ has how many solutions mod 50? (A) 0 (B) 1 (C) 5 (D) 15
- In Miller–Rabin for $n = 21$, $a = 2$ the test says: (A) prime (B) composite (C) Carmichael (D) undefined
- Best-case time of insertion sort: (A) $O(1)$ (B) $O(n)$ (C) $O(n \log n)$ (D) $O(n^2)$
- Minimum comparisons of divide-and-conquer max–min for $n = 8$: (A) 14 (B) 12 (C) 10 (D) 8
- Worst case of quick sort with last-element pivot occurs for: (A) random data (B) already sorted data (C) all distinct halves (D) none
- Building a heap of n elements takes: (A) $O(\log n)$ (B) $O(n)$ (C) $O(n \log n)$ (D) $O(n^2)$
- Worst-case linear selection uses groups of: (A) 2 (B) 3 (C) 5 (D) $\log n$

16. Fractional knapsack is solved optimally by sorting items on: (A) weight (B) profit (C) profit/weight (D) weight/profit
17. Job sequencing, $P = (3, 5, 20, 18, 1, 6, 30)$, $D = (1, 3, 4, 3, 2, 1, 2)$: maximum profit is: (A) 62 (B) 74 (C) 83 (D) 69
18. MST of a graph with $|V|$ vertices has exactly: (A) $|V|$ edges (B) $|V| - 1$ edges (C) $|V| + 1$ edges (D) $|E|$ edges
19. Dijkstra's algorithm fails when the graph has: (A) cycles (B) negative edge weights (C) parallel paths (D) directed edges
20. Floyd–Warshall runs in: (A) $O(n^2)$ (B) $O(n^3)$ (C) $O(n^2 \log n)$ (D) $O(2^n)$
21. Edit distance between “kitten” and “sitting” is: (A) 2 (B) 3 (C) 4 (D) 5
22. Matrix chain with $p = (10, 20, 30, 40, 30)$: minimum multiplications: (A) 51000 (B) 54000 (C) 30000 (D) 18000
23. Held–Karp TSP time complexity: (A) $O(n!)$ (B) $O(n^2 2^n)$ (C) $O(n^3)$ (D) $O(2^n)$ with space $O(n^2)$
24. Johnson's rule schedules two-machine flow shop in: (A) $O(n)$ (B) $O(n \log n)$ (C) $O(n^2)$ (D) $O(2^n)$
25. Number of solutions of 4-Queens: (A) 1 (B) 2 (C) 4 (D) 0
26. The bounding conditions for sum of subsets are: (A) $sum > M$ or $sum + rem < M$ (B) $sum < M$ (C) $rem > M$ (D) none
27. Time complexity of graph colouring by backtracking with m colours: (A) $O(V \cdot m^V)$ (B) $O(m^2 V)$ (C) $O(V!)$ (D) $O(V^2)$
28. Optimal profit of 0/1 knapsack $w = (2, 3, 4, 5)$, $p = (3, 5, 6, 10)$, $W = 8$: (A) 11 (B) 13 (C) 15 (D) 16
29. The class NP-complete equals: (A) $P \cap NP$ (B) $NP \cap NP\text{-hard}$ (C) $NP \cup NP\text{-hard}$ (D) $NP\text{-hard} \setminus NP$
30. The first problem proved NP-complete (Cook's theorem) is: (A) Clique (B) TSP (C) SAT (D) Vertex cover
31. APPROX-VERTEX-COVER has approximation ratio: (A) 1 (B) 1.5 (C) 2 (D) $\log n$

Answer key

1	B	2	C	3	B	4	C	5	B
6	A	7	B	8	D	9	C	10	B
11	B	12	C	13	B	14	B	15	C
16	B	17	B	18	B	19	B	20	B
21	C	22	B	23	B	24	B	25	A
26	A	27	C	28	B	29	C	30	C